

Bootstrapping (T)FHE Ciphertexts via Automorphisms: Closing the Gap Between Binary and Gaussian Keys^{*}

Olivier Bernard  and Marc Joye 

Zama, Paris, France

`firstname.lastname@zama.ai`

Abstract. The GINX method in TFHE enables low-latency ciphertext bootstrapping with relatively small bootstrapping keys but is limited to binary or ternary key distributions. In contrast, the AP method supports arbitrary key distributions, albeit at the cost of significantly larger bootstrapping keys. Building on AP, automorphism-based methods, introduced in LMK⁺ (EUROCRYPT 2023), achieve smaller key sizes. However, each automorphism application necessitates a key switch, introducing additional computational overhead and noise accumulation.

This paper advances automorphism-based methods in two important ways. First, it proposes a novel traversal blind rotation algorithm that optimizes the number of key switches for a given key material. Second, it introduces a new external product that is automorphism-parametrized and seamlessly applies an automorphism to one of the input ciphertexts. Together, these techniques substantially reduce the number of key switches, resulting in faster bootstrapping and improved noise control. As an independent contribution, we introduce a comprehensive theoretical framework for analyzing the expected number of automorphism key switches. The predictions of this framework perfectly align with the results of extensive numerical experiments, demonstrating its practical relevance.

In typical settings, by leveraging additional key material, the LLW⁺ approach (TCHES 2024) reduces the number of key switches by 17% compared to LMK⁺. Our combined techniques achieve a 46% reduction using similar key material and can eliminate an arbitrary large number (e.g., more than 99%) of key switches with only a moderate (9×) increase in key material size. As a result, the total bootstrapping runtime is decreased by more than 34%.

Keywords: Fully homomorphic encryption (FHE) · Ciphertext bootstrapping · Blind rotation · Automorphisms · Implementation

1 Introduction

Fully homomorphic encryption (FHE) schemes [RAD78, Gen10] enable the evaluation of any circuit over encrypted data, ensuring that the data remains end-to-

^{*} © IACR 2025. Minor revision of the paper published in the Proceedings of ASIACRYPT 2025, Part VII, LNCS 16251, Springer, [doi:10.1007/978-981-95-5122-4_1](https://doi.org/10.1007/978-981-95-5122-4_1).

end encrypted without requiring decryption for processing. Since their inception in 2009, substantial research has focused on enhancing the practical efficiency of FHE, which remains a critical challenge for its widespread deployment.

Current FHE implementations require noisy ciphertexts for their security. However, as homomorphic operations are performed, noise accumulates, and beyond a certain threshold, ciphertexts become undecryptable. This issue is resolved *via* bootstrapping, a technique that refreshes ciphertexts by reducing the noise to an acceptable level through a homomorphic evaluation of the decryption algorithm [Gen10]. On input a (highly) noisy ciphertext, the output is a ciphertext encrypting the same message, but with a reduced level of noise. The most efficient instantiations of ciphertext bootstrapping makes use of an astute technique known as blind rotation [AP14, DM15, GINX16, CGGI20].

Using a polynomial representation, the blind rotation consists in the homomorphic evaluation of $v(x) \cdot x^{\sum_{i=1}^n a_i s_i}$ for some polynomial v , where the vector $\mathbf{a} = (a_1, \dots, a_n)$ is public and the vector $\mathbf{s} = (s_1, \dots, s_n)$ is secret. The blind rotation can also be used as a convenient way to implement a homomorphic look-up table with polynomials; see e.g., [Joy22]. When coupled with a bootstrapping operation, the homomorphic evaluation of a look-up table is also known as programmable bootstrapping [CJP21]. Since any univariate function (over a small domain) can always be expressed as a look-up table, an encryption scheme allowing homomorphic addition and homomorphic table look-up turns out to be fully homomorphic. Regular bootstrapping, whose primary goal is to reduce the noise, corresponds to a programmable bootstrapping for the identity map.

Several approaches are known for the blind rotation, with different trade-offs between vector coefficients, evaluation key material, and number of operations:

1. The GINX algorithm [GINX16, CGGI20] features comparatively small evaluation key material associated to secret vector $\mathbf{s}$, but requires $\mathbf{s}$ to be *binary*.
2. The AP algorithm [AP14, DM15] requires a huge amount of evaluation key material associated to secret vector $\mathbf{s}$ (exponential in the coefficient space of public vectors $\mathbf{a}$), although imposing no restrictions on the distribution of $\mathbf{s}$.
3. Automorphism-based algorithms [BDF18, LMK⁺23, WWL⁺24], ultimately based on AP, decrease the global amount of evaluation key material thanks to homomorphic evaluations of automorphisms. However, each such automorphism evaluation requires a key switch, which *in fine* leads to a substantial increase in the overall computational cost and the noise growth. Even in the most favorable case, one automorphism key switch operation costs at least as much as 0.6 external products and has almost the same noise growth.

Previous attempts to enhance GINX-type algorithms for using other distributions of the secret vector coefficients conclude to limit keys to small distributions (typically, binary or ternary) in order to avoid a blow-up of evaluation key material, thereby reducing their applicability; see e.g., [JP22]. Certain adaptations and extensions of the AP algorithm to various encryption schemes, such as NTRU, can be found in [XZD⁺23, MKMS24, LLW⁺24]. In order to optimize the blind rotation, automorphism-based variants of AP focus on reducing

the computational overhead introduced by the key switches incurred from the homomorphic evaluation of automorphisms [LLW⁺24, Lee24, ZWC25].

External products and automorphisms The external product of ciphertexts is a fundamental operation in fully homomorphic encryption, enabling the blind rotation for (programmable) bootstrapping, and also playing a critical role in advanced techniques such as batched bootstrapping [MKMS24] and circuit bootstrapping [WWL⁺24]. Specifically, given a ciphertext $c \leftarrow \text{Enc}_{\text{sk}}(\mu)$ encrypting a plaintext μ under the algorithm Enc , and another (extended) ciphertext $\bar{c} \leftarrow \text{Enc}_{\text{sk}}^{\otimes}(\bar{\mu})$ encrypting a plaintext $\bar{\mu}$ under an associated algorithm $\text{Enc}^{\otimes}$, the external product operation, denoted $\otimes$, produces a new ciphertext $c' = c \otimes \bar{c}$, which encrypts the product of the plaintexts, $\mu' = \mu \cdot \bar{\mu}$.

A crucial observation is that in all of the automorphism-based methods, the homomorphic evaluation of automorphisms, including the associated key switch, is most of the time combined with an external product. Specifically, these methods require the encryption of $\psi(\mu) \cdot \bar{\mu}$ from the encryptions of μ and of $\bar{\mu}$, for some automorphism ψ . This process is typically carried out in three sequential steps, beginning with the ciphertexts $c \leftarrow \text{Enc}_{\text{sk}}(\mu)$ and $\bar{c} \leftarrow \text{Enc}_{\text{sk}}^{\otimes}(\bar{\mu})$. First, the automorphism ψ is applied to c , resulting in $c_1 \leftarrow \psi(c) \in \text{Enc}_{\psi(\text{sk})}(\psi(\mu))$. Next, a key switch (KS) operation is performed to transform c_1 into $c_2 \leftarrow \text{KS}_{\psi(\text{sk}) \rightarrow \text{sk}}(c_1)$, which belongs to $\text{Enc}_{\text{sk}}(\psi(\mu))$. Finally, the transformed ciphertext c_2 is combined with $\bar{c}$ through an external product operation, yielding $c_3 \leftarrow c_2 \otimes \bar{c}$, which encrypts the product $\psi(\mu) \cdot \bar{\mu}$, i.e., $c_3 \leftarrow \text{Enc}_{\text{sk}}(\psi(\mu) \cdot \bar{\mu})$.

Our techniques and results The main contribution of this paper is the introduction of a novel operation for FHE, the *Automorphism-Parametrized External Product*. This operation integrates three key steps—automorphism evaluation, its associated key switch, and an external product—at the computational cost of a *single* external product. Specifically, we define this new operator as follows:

$$\text{Enc}_{\text{sk}}(\mu) \otimes_{\psi} \text{Enc}_{\text{sk}}^{\otimes, \psi}(\bar{\mu}) \leftarrow \text{Enc}_{\text{sk}}(\psi(\mu) \cdot \bar{\mu}),$$

where $\text{Enc}_{\text{sk}}(\mu) \otimes_{\psi} \text{Enc}_{\text{sk}}^{\otimes, \psi}(\bar{\mu})$ is computed as $\psi(\text{Enc}_{\text{sk}}(\mu)) \otimes \text{Enc}_{\text{sk}}^{\otimes, \psi}(\bar{\mu})$. Here, $\text{Enc}_{\text{sk}}^{\otimes, \psi}(\bar{\mu})$ is a newly introduced ciphertext format, which we call *automorphism-extended* ciphertext. As will become evident in Section 4, this format naturally arises from the associated encryption algorithm $\text{Enc}^{\otimes}$; in fact, both formats coincide when the secret key is $\psi(\text{sk})$ instead of sk . Notably, our new operator eliminates the key switch overhead, which brings a significant practical advantage by substantially reducing the associated computational cost and noise growth.

It is also important to note that although the format of the second ciphertext, $\text{Enc}_{\text{sk}}^{\otimes, \psi}(\bar{\mu})$, has been modified, our new external product still outputs a regular Enc_{sk} ciphertext as before. This ensures seamless integration in most applications that rely on the product in the exponent based on automorphisms, including blind rotation. Indeed, in such cases the second ciphertext is typically a global input (e.g., a bootstrapping key in the context of blind rotation).

Another contribution of this paper is an improved algorithm for blind rotation in FHE, termed the *Traversal Windowed-Horner Method* and detailed in [Algorithm 3.2](#). Building on Algorithm 7 of [\[LMK⁺23\]](#), which has been reformulated in [Algorithm 3.1](#) for clarity and easier adaptation, our new approach addresses gaps between two non-empty sets of mask values by incorporating sign changes directly into automorphism evaluations. This integration reduces the average gap size between automorphism applications, enhancing efficiency, particularly for smaller window sizes. Compared to the original method, our traversal algorithm achieves consistent reductions in the number of automorphism key switches, with typical gains of up to 8% depending on the parameters, whereas both methods converge to comparable efficiency for larger windows.

Our second main contribution is a new very effective automorphism-based blind rotation, the so-called *$\mathcal{S}$ -Parametrized Method* formalized in [Algorithm 4.1](#), which leverages the power of our automorphism-parametrized external product inside the workflow of our traversal method. Depending on a set $\mathcal{S}$ of admissible “shortcut” automorphisms to use with the new parametrized external product, it provides flexible trade-offs between key size, which is linearly linked to $\#\mathcal{S}$, and performance and noise growth, which both improve when $\#\mathcal{S}$ grows since this lowers the number of required key switches.

To illustrate the benefits of this technique, we experimentally evaluate its impact on several parameter sets: (i) a parameter set from [\[LMK⁺23, LLW⁺24\]](#), designed for Gaussian keys and boolean messages, allowing direct comparisons with previous works; (ii) a parameter set specified in the `TFHE-rs` library [\[Zam22\]](#), primarily tailored for binary keys and 4-bit payloads, reflecting practical application scenarios. Extensive results are given in [Table 5.4](#). With only a moderate increase in bootstrapping key sizes similar to [\[LLW⁺24, Lee24\]](#), we reduce the number of key switches compared to [\[LMK⁺23\]](#) by approximately 49.1% (resp. 46.4%), significantly surpassing (an adaptation of) [\[LLW⁺24\]](#), which would achieve only an 18.5% (resp. 17.0%) reduction. By allowing a slightly larger increase in bootstrapping key sizes, we achieve a 59.4% (resp. 55.5%) reduction in key switches relatively to [\[LLW⁺24\]](#) and 66.9% (resp. 63.1%) compared to [\[LMK⁺23\]](#). Moreover, the number of key switches drops to as few as 2 or 3 on average for keys that are only 9 times larger than in [\[LMK⁺23\]](#), resulting in a total runtime improvement by more than 34%. This outperforms AP bootstrapping by far, which for comparable performance requires keys that are more than 2 orders of magnitude larger. Note the additional keys can easily be prefetched according to the mask components, minimizing the impact on memory bandwidth.

Finally, as a last contribution, we provide a thorough analysis of the complexity and noise growth of automorphism-based blind rotation algorithms. We develop a theoretical framework that reduces this problem to studying the distribution of gaps in random divisions of an interval. Our new automorphism-parametrized method thus provably demonstrates fewer key switches and improved efficiency compared to existing approaches, and numerical experiments validate these improvements. In summary, our new techniques not only offer substantial reductions in key-switching overhead but also provide flexible trade-offs

to meet varying performance and resource requirements. Although primarily presented for the TFHE cryptosystem, our methods and techniques are adaptable to other FHE schemes, such as FHEW [DM15] and FINAL [BIP⁺22], offering versatile and practical improvements for bootstrapping ciphertexts in FHE.

Outline of the paper Section 2 reviews the relevant background. Section 3 reformulates and improves the [LMK⁺23] automorphism-based blind rotation. Our automorphism-parametrized external product is introduced in Section 4 and we thereafter formally describe our most efficient $\mathcal{S}$ -parametrized blind rotation. Finally, Section 5 contains our theoretical framework and numerical experiments.

2 Definitions and Notations

Let $q, t < q$, and k be positive integers, and let $\Delta = \lfloor \frac{q}{t} \rfloor$. The m -th cyclotomic polynomial Φ_m defines the cyclotomic field of conductor m , for $m \not\equiv 2 \pmod{4}$, as $\mathcal{K} = \mathbb{Q}[x]/\langle \Phi_m(x) \rangle$. The degree of Φ_m is $N = \varphi(m)$, where φ is Euler's totient function. Common values for m include m a power of two, a prime $p > 2$, or of the form $p^k, 4p^k, 2^a 3^b$ [JW22]. Let also $\mathcal{R} = \mathbb{Z}[x]/\langle \Phi_m(x) \rangle$ be the ring of integers of $\mathcal{K}$, and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$. The Galois group of $\mathcal{K}/\mathbb{Q}$ is isomorphic to $(\mathbb{Z}/m\mathbb{Z})^\times$ and consists of automorphisms τ_u defined by $\tau_u(x) = x^u$, for $u \in (\mathbb{Z}/m\mathbb{Z})^\times$; the identity automorphism τ_1 is also denoted as id .

GLWE ciphertexts GLWE stands for *generalized-LWE* and LWE refers to *Learning with Errors* [Reg09]. GLWE-type encryption appears for example in [SSTX09, LPR10, LS15]. Cleartext messages in a GLWE encryption scheme are polynomials in $\mathcal{R}$ with coefficients modulo t . Prior to encryption, a cleartext message m is first encoded as a plaintext $\mu = \Delta \cdot m \in \mathcal{R}_q$. The GLWE encryption of $\mu \in \mathcal{R}_q$ under private key $\mathfrak{z} = (z_1, \dots, z_k) \in \mathcal{R}^k$ is then given by

$$\text{GLWE}_{\mathfrak{z}}(\mu) \leftarrow (a_1, \dots, a_k, \mathfrak{b} = \sum_{j=1}^k a_j \cdot z_j + \mu + e) \in \mathcal{R}_q^{k+1},$$

where $a_1, \dots, a_k$ are random polynomials sampled in $\mathcal{R}_q$ and $e \in \mathcal{R}$ is a random noise polynomial with small coefficients. Vector $(a_1, \dots, a_k)$ is called the *mask*, $\mathfrak{b}$ is the *body*, and the corresponding *error* e is denoted by $\text{Err}(\text{GLWE}_{\mathfrak{z}}(\mu))$.

Gadget-GLWE ciphertexts and extended-GLWE ciphertexts Following the presentation of [MP21], the simplest way to view extended-GLWE ciphertexts¹ is through gadget decomposition of GLWE ciphertexts.

Applied to $\ell \in \mathcal{R}_q$, the gadget decomposition of ℓ with respect to the gadget vector $\mathbf{g} = (g_1, \dots, g_\ell) \in \mathcal{R}_q^\ell$ is given by a vector $\nabla \ell \in \mathcal{R}^\ell$ s.t. $\langle \nabla \ell, \mathbf{g} \rangle \approx \ell$. The *quality* of the gadget decomposition is defined by $\beta_\nabla := \|\nabla \ell\|_\infty$, and its *precision* is given by $\varepsilon_\nabla := \|\ell - \langle \nabla \ell, \mathbf{g} \rangle\|_\infty$; see e.g., [CGGI20, BJ24]. The

¹ Also known as GGSW ciphertexts, which generalize the GSW encryption [GSW13].

corresponding *gadget-GLWE ciphertext* (indicated with a ∇ superscript) of a plaintext $\bar{\mu} \in \mathcal{R}_q$ under private key $\mathfrak{z} = (\mathfrak{z}_1, \dots, \mathfrak{z}_k) \in \mathcal{R}^k$ is defined as

$$\text{GLWE}_{\mathfrak{z}}^{\nabla}(\bar{\mu}) \leftarrow (\text{GLWE}_{\mathfrak{z}}(\mathfrak{g}_1 \cdot \bar{\mu}), \dots, \text{GLWE}_{\mathfrak{z}}(\mathfrak{g}_{\ell} \cdot \bar{\mu})) .$$

This leveled encryption is used to build an *extended-GLWE ciphertext*

$$\text{GLWE}_{\mathfrak{z}}^{\circledast}(\bar{\mu}) \leftarrow (\text{GLWE}_{\mathfrak{z}}^{\nabla}(-\mathfrak{z}_1 \cdot \bar{\mu}), \dots, \text{GLWE}_{\mathfrak{z}}^{\nabla}(-\mathfrak{z}_k \cdot \bar{\mu}), \text{GLWE}_{\mathfrak{z}}^{\nabla}(\bar{\mu})) ,$$

whose definition coincides with the definition of a GGSW ciphertext.

External product A GLWE ciphertext $\text{GLWE}_{\mathfrak{z}}(\mu)$ can be combined with an extended GLWE ciphertext $\text{GLWE}_{\mathfrak{z}}^{\circledast}(\bar{\mu})$ to yield another GLWE ciphertext through the *external product*, which is denoted by $\circledast$ and explicitly defined by

$$\begin{aligned} \text{GLWE}_{\mathfrak{z}}(\mu) \circledast \text{GLWE}_{\mathfrak{z}}^{\circledast}(\bar{\mu}) := \\ \langle \nabla \mathfrak{t}, \text{GLWE}_{\mathfrak{z}}^{\nabla}(\bar{\mu}) \rangle + \sum_{j=1}^k \langle \nabla \mathfrak{a}_j, \text{GLWE}_{\mathfrak{z}}^{\nabla}(-\mathfrak{z}_j \cdot \bar{\mu}) \rangle . \end{aligned} \quad (2.1)$$

In certain cases (e.g., [BCG⁺24, Theorem 3]), different gadget decomposition levels are used for the mask and the body of the ciphertext, denoted respectively by ℓ_1 and ℓ_2 . This is indicated by writing the individual decompositions as ∇_{ℓ_1} and ∇_{ℓ_2} , with the overall decomposition given by $\nabla = \nabla_{\ell_1, \ell_2}$.

It can be verified that $\text{GLWE}_{\mathfrak{z}}(\mu) \circledast \text{GLWE}_{\mathfrak{z}}^{\circledast}(\bar{\mu}) \leftarrow \text{GLWE}_{\mathfrak{z}}(\mu \cdot \bar{\mu})$, provided that (i) the gadget decomposition is sufficiently exact and (ii) $e \cdot \bar{\mu}$ is sufficiently small, where $e = \text{Err}(\text{GLWE}_{\mathfrak{z}}(\mu))$ [CGGI20, Theorem 3.13 and Corollary 3.14].

Proposition 2.1. *Assume m is a power of two. Let $\nabla = \nabla_{\ell_1, \ell_2}$ be a gadget decomposition of quality $\beta_{\nabla} = (\beta_1, \beta_2)$ and precision $\varepsilon_{\nabla} = (\varepsilon_1, \varepsilon_2)$, whose output values are uniform and centered around 0. Let e_{in} and $\bar{e}$ represent the error associated with valid samples $\text{GLWE}_{\mathfrak{z}}(\mu)$ and $\text{GLWE}_{\mathfrak{z}}^{\circledast}(\bar{\mu})$, respectively. Then, $\text{GLWE}_{\mathfrak{z}}(\mu) \circledast \text{GLWE}_{\mathfrak{z}}^{\circledast}(\bar{\mu})$ is a sample of $\text{GLWE}_{\mathfrak{z}}(\mu \cdot \bar{\mu})$ with an error $\mathcal{E}$ of variance*

$$\sigma_{\circledast}^2 \leq \|\bar{\mu}\|_2^2 \cdot \sigma_{\text{in}}^2 + N \left(\ell_2 \frac{\beta_2^2}{12} + k \ell_1 \frac{\beta_1^2}{12} \right) \cdot \sigma_{\nabla}^2 + \|\bar{\mu}\|_2^2 \left(\frac{\varepsilon_2^2}{12} + k N \frac{\varepsilon_1^2}{12} \cdot \mathbb{E}[\mathfrak{z}_{j,i}^2] \right) .$$

Proof. This is a special case of the generalized result presented in Proposition 4.3, obtained by taking $\psi = \text{id}$ and $C_{\infty} = 1$. In particular, the exact expression of the error term $\mathcal{E} = \mathcal{B} - \langle \mathfrak{A}, \mathfrak{z} \rangle - \mu \cdot \bar{\mu}$ is given by

$$\begin{aligned} \mathcal{E} = \bar{\mu} \cdot e_{\text{in}} + \left(\langle \nabla_{\ell_2} \mathfrak{t}, \bar{e}_0 \rangle + \sum_{1 \leq j \leq k} \langle \nabla_{\ell_1} \mathfrak{a}_j, \bar{e}_j \rangle \right) \\ + \bar{\mu} \cdot \left(e_{\nabla_{\ell_2}}(\mathfrak{t}) - \sum_{1 \leq j \leq k} \mathfrak{z}_j \cdot e_{\nabla_{\ell_1}}(\mathfrak{a}_j) \right) , \end{aligned}$$

where $e_{\nabla_{\ell_u}}(w) := \langle \nabla_{\ell_u} w, \mathfrak{g}_u \rangle - w$ for $u \in \{1, 2\}$ and any $w \in \mathcal{R}_q$. $\square$

Homomorphic evaluation of automorphisms Given an automorphism $\tau_u: x \mapsto x^u$ for some unit $u \in (\mathbb{Z}/m\mathbb{Z})^\times$ and $\text{GLWE}_{\mathfrak{z}}(\mu) \leftarrow (\mathfrak{a}_1, \dots, \mathfrak{a}_k, \mathfrak{t}) \in \mathcal{R}_q^{k+1}$, a GLWE ciphertext of $\mu := \mu(x) \in \mathcal{R}_q$ under private key $\mathfrak{z} = (\mathfrak{z}_1, \dots, \mathfrak{z}_k)$, observe that

$$\tau_u(\text{GLWE}_{\mathfrak{z}}(\mu)) \leftarrow (\tau_u(\mathfrak{a}_1), \dots, \tau_u(\mathfrak{a}_k), \tau_u(\mathfrak{t})) \in \text{GLWE}_{\tau_u(\mathfrak{z})}(\tau_u(\mu))$$

is a GLWE encryption of $\tau_u(\mu) = \mu(x^u)$ under key $\tau_u(\mathfrak{z}) = (\tau_u(\mathfrak{z}_1), \dots, \tau_u(\mathfrak{z}_k))$, provided that $\tau_u(e)$, where $e = \text{Err}(\text{GLWE}_{\mathfrak{z}}(\mu))$, stays sufficiently small. Now, let $\text{ak}_u := \text{ksk}_{\tau_u(\mathfrak{z}) \rightarrow \mathfrak{z}}$ be an *automorphism key*, i.e., a key-switching key that allows converting a ciphertext under key $\tau_u(\mathfrak{z})$ back to a ciphertext under key $\mathfrak{z}$. Specifically, ak_u is defined as $\text{ak}_u \leftarrow (\text{GLWE}_{\mathfrak{z}}^\nabla(-\tau_u(\mathfrak{z}_1)), \dots, \text{GLWE}_{\mathfrak{z}}^\nabla(-\tau_u(\mathfrak{z}_k)))$. Then, a subsequent key switch KS_{ak_u} on $\tau_u(\text{GLWE}_{\mathfrak{z}}(\mu))$ with ak_u yields the ciphertext $\text{GLWE}_{\mathfrak{z}}(\tau_u(\mu))$. Thus, the composition of these two operations homomorphically evaluates τ_u and will be denoted by $\text{HomAut}_u(\cdot, \text{ak}_u)$; explicitly,

$$\begin{aligned} \text{HomAut}_u(\text{GLWE}_{\mathfrak{z}}(\mu), \text{ak}_u) &\leftarrow \text{KS}_{\text{ak}_u} \circ \tau_u(\text{GLWE}_{\mathfrak{z}}(\mu)) := \\ &(0, \dots, 0, \tau_u(\mathfrak{t})) + \sum_{j=1}^k \langle \nabla \tau_u(\mathfrak{a}_j), \text{GLWE}_{\mathfrak{z}}^\nabla(-\tau_u(\mathfrak{z}_j)) \rangle. \end{aligned} \quad (2.2)$$

Correctness supposes that the resulting noise keeps below a certain threshold.

Proposition 2.2. *Assume m is a power of two. Let $\nabla_{\ell_{\text{ks}}}$ be a gadget decomposition of quality β_{ks} and precision ε_{ks} , whose output values are uniform and centered around 0. Let e_{in} and $\bar{\mathbf{e}}_{\text{ks},j}$, where $j \in \llbracket 1, k \rrbracket$, represent the error associated with respectively valid samples $\text{GLWE}_{\mathfrak{z}}(\mu)$ and $\text{GLWE}_{\mathfrak{z}}^\nabla(-\tau_u(\mathfrak{z}_j))$. Then, $\text{HomAut}_u(\text{GLWE}_{\mathfrak{z}}(\mu), \text{ak}_u)$ is a sample of $\text{GLWE}_{\mathfrak{z}}(\tau_u(\mu))$ with error of variance*

$$\sigma_{\text{aut}}^2 \leq \sigma_{\text{in}}^2 + N \left(k \ell_{\text{ks}} \frac{\beta_{\text{ks}}^2}{12} \right) \cdot \sigma_{\text{ks}}^2 + kN \left(\mathbb{E}[\mathfrak{z}^2] \cdot \frac{\varepsilon_{\text{ks}}^2}{12} \right).$$

Proof. The exact expression of the error term $\mathcal{E} = \mathcal{B} - \langle \mathfrak{a}, \mathfrak{z} \rangle - \tau_u(\mu)$ is given by

$$\mathcal{E} = \tau_u(e_{\text{in}}) + \sum_{1 \leq j \leq k} \langle \nabla_{\ell_{\text{ks}}} \tau_u(\mathfrak{a}_j), \bar{\mathbf{e}}_{\text{ks},j} \rangle - \sum_{1 \leq j \leq k} \tau_u(\mathfrak{z}_j) \cdot e_{\nabla_{\ell_{\text{ks}}}}(\tau_u(\mathfrak{a}_j)),$$

where $e_{\nabla_{\ell_{\text{ks}}}}(w) := \langle \nabla_{\ell_{\text{ks}}} w, \mathbf{g}_{\text{ks}} \rangle - w$ for any $w \in \mathcal{R}_q$. $\square$

We emphasize that an automorphism key switch has comparable computational cost and noise growth than an external product. The noise growth ratio for both operations directly stems from [Propositions 2.1](#) and [2.2](#) and is relatively close to 1 for comparable decomposition parameters. As for the computational costs, counting (inverse) transforms (from) to the Fourier domain, assuming $\ell = \ell_1 = \ell_2 = \ell_{\text{ks}}$ for simplicity's sake, the ratio between an automorphism key switch and an external product is given by $\frac{1+k(\ell+1)}{(k+1)(\ell+1)}$, which lies between $\frac{k}{k+1}$ and 1. For typical values $k = 1$ and $\ell \leq 3$, one automorphism key switch thus computationally weights at least as much as $\frac{5}{8}$ external products. This $k = 1$ case is actually the most favorable, as k increases, $\frac{k}{k+1}$ approaches 1.

AP blind rotation An important application of the external product is the evaluation of an inner product in the exponent, or the related task of performing a *blind rotation*. Given an LWE ciphertext $\tilde{\mathbf{c}} = (\tilde{a}_1, \dots, \tilde{a}_n, \tilde{b}) \in (\mathbb{Z}/m\mathbb{Z})^{n+1}$ under a private key $\mathbf{s} = (s_1, \dots, s_n)$ and a so-called test polynomial $\mathbf{v} \in \mathcal{R}_q$, the blind rotation consists in evaluating homomorphically $\mathbf{v}(x) \cdot x^{-\tilde{b} + \sum_{i=1}^n \tilde{a}_i s_i}$. Additional key material known as bootstrapping keys is made available for the computation, namely the encryption of the key digits $s_1, \dots, s_n$. In its generic version, the AP blind rotation requires a set of $n(m-1)$ bootstrapping keys,

$$\text{bsk}^{\text{AP}} := \left\{ \text{bsk}^{\text{AP}}[i, u] \leftarrow \text{GLWE}_{\mathfrak{d}}^{\otimes}(x^{u s_i}) \mid i \in \llbracket 1, n \rrbracket \text{ and } u \in \llbracket 1, m-1 \rrbracket \right\}.$$

The AP method uses an accumulator $\text{ACC} \in \mathcal{R}_q^{k+1}$ that successively contains a GLWE encryption of $\mathbf{q}_0 = \mathbf{v} \cdot x^{-\tilde{b}}$, and then $\mathbf{q}_i \leftarrow \mathbf{q}_{i-1} \cdot x^{\tilde{a}_i s_i} = \mathbf{v} \cdot x^{-\tilde{b} + \sum_{j=1}^i \tilde{a}_j s_j}$ for $i \in \llbracket 1, n \rrbracket$. At the end of the iteration, the accumulator indeed contains a GLWE encryption of $\mathbf{v} \cdot x^{-\tilde{b} + \sum_{i=1}^n \tilde{a}_i s_i}$. In an algorithmic form, this writes as:

```

ACC  $\leftarrow$   $(0, \dots, 0, x^{-\tilde{b}} \cdot \mathbf{v})$  /* Trivial GLWE encryption of  $\mathbf{q}_0$  with null noise */
for  $i = 1$  to  $n$  do
  if  $\tilde{a}_i \neq 0$  then ACC  $\leftarrow$  ACC  $\otimes$   $\text{bsk}^{\text{AP}}[i, \tilde{a}_i]$ 
return ACC

```

Automorphism-based methods The use of automorphisms aims at reducing the size of the additional key material in the blind rotation while containing the computational overhead. Without loss of generality, automorphism-based methods assume that each mask component $\tilde{a}_i$ of the input LWE ciphertext, $i \in \llbracket 1, n \rrbracket$, is either 0 or belongs to $(\mathbb{Z}/m\mathbb{Z})^\times$, so that when non-zero, each indeed corresponds to an automorphism $\tau_{\tilde{a}_i} : x \mapsto x^{\tilde{a}_i}$. Different strategies from [BDF18, MKMS24, LMK⁺23, WWL⁺24] are summarized in Appendix A to reduce to this setting.

As described above, a loop iteration of the AP blind rotation consists in computing a GLWE ciphertext $\mathbf{c}^{(i)}$ of $\mathbf{q}_i(x) = \mathbf{q}_{i-1}(x) \cdot x^{\tilde{a}_i s_i}$ from a GLWE ciphertext $\mathbf{c}^{(i-1)}$ of $\mathbf{q}_{i-1}(x)$. Using automorphisms as in [BDF18, Algorithms 5 and 6],² this can be achieved in three consecutive steps as

$$\begin{aligned}
(1) \quad \mathbf{c}^{(i)} &\leftarrow \text{HomAut}_{1/\tilde{a}_i}(\mathbf{c}^{(i-1)}, \text{ak}_{1/\tilde{a}_i}) \in \text{GLWE}_{\mathfrak{d}}(\mathbf{q}_{i-1}(x^{1/\tilde{a}_i})) \\
(2) \quad \mathbf{c}^{(i)} &\leftarrow \mathbf{c}^{(i)} \otimes \text{GLWE}_{\mathfrak{d}}^{\otimes}(x^{s_i}) \in \text{GLWE}_{\mathfrak{d}}(\mathbf{q}_{i-1}(x^{1/\tilde{a}_i}) \cdot x^{s_i}) \\
(3) \quad \mathbf{c}^{(i)} &\leftarrow \text{HomAut}_{\tilde{a}_i}(\mathbf{c}^{(i)}, \text{ak}_{\tilde{a}_i}) \in \text{GLWE}_{\mathfrak{d}}(\mathbf{q}_{i-1}(x) \cdot x^{\tilde{a}_i s_i})
\end{aligned}$$

where the inverses are taken modulo m . The resulting blind rotation algorithm (depicted in Figure 2.1(a)) requires the following key material:

$$\text{bsk}^{\text{AUT}} := \left\{ \text{bsk}^{\text{AUT}}[i] \leftarrow \text{GLWE}_{\mathfrak{d}}^{\otimes}(x^{s_i}) \mid i \in \llbracket 1, n \rrbracket \right\} \quad (2.3)$$

and

$$\text{ak}^{\text{AUT}} := \left\{ \text{ak}^{\text{AUT}}[u] \leftarrow \text{ksk}_{\tau_u(\mathfrak{d}) \rightarrow \mathfrak{d}} \mid u \in (\mathbb{Z}/m\mathbb{Z})^\times \setminus \{1\} \right\}. \quad (2.4)$$

² Originally in the circulant setting, however conceptually it remains exactly the same.

<pre> ACC ← (0, ..., 0, v · x^{-b̃}) for i ∈ [1, n] such that $\tilde{a}_i \neq 0$ do u ← 1/$\tilde{a}_i$ mod m ACC ← HomAut_u(ACC, ak^{AUT}[u]) ACC ← ACC ⊗ bsk^{AUT}[i] ACC ← HomAut_{$\tilde{a}_i$}(ACC, ak^{AUT}[$\tilde{a}_i$]) return ACC </pre>	<pre> ACC ← (0, ..., 0, v · x^{-b̃}), $\tilde{a}_{\text{old}} \leftarrow 1$ for i ∈ [1, n] such that $\tilde{a}_i \neq 0$ do u ← $\tilde{a}_{\text{old}}/\tilde{a}_i$ mod m, $\tilde{a}_{\text{old}} \leftarrow \tilde{a}_i$ ACC ← HomAut_u(ACC, ak^{AUT}[u]) ACC ← ACC ⊗ bsk^{AUT}[i] ACC ← HomAut_{$\tilde{a}_{\text{old}}$}(ACC, ak^{AUT}[$\tilde{a}_{\text{old}}$]) return ACC </pre>
(a) Basic method.	(b) Telescoping method.

Fig. 2.1: Automorphism-based methods.

The number of homomorphic automorphism evaluations must not be overlooked as each involves a key switch. As noted e.g., in [XZD⁺23, Algorithm 1] or [MKMS24, Algorithm 2, inner loop], the two automorphisms of each loop iteration can be combined together, halving the number of required key switches. This trick is applied and detailed in Figure 2.1(b) (telescoping method). When $u = 1$, HomAut_u is the identity map and so is always skipped. Hence, the $\tilde{a}_i$'s can be regrouped by values, so that if the set $\{\tilde{a}_i \mid i \in [1, n]\} \setminus \{0, 1\}$ has cardinality α , at most $\alpha < \varphi(m)$ automorphism evaluations and key switches are performed.

Compared to the generic, non automorphism-based AP blind rotation presented earlier, the telescoping method already behaves much nicer. The key material drops from $n(m-1)$ GLWE[®] ciphertexts (i.e., $n(k+1)(m-1)$ GLWE[∇] ciphertexts) to n GLWE[®] ciphertexts plus $k(\varphi(m) - 1)$ GLWE[∇] ciphertexts. Computation-wise, at most $\min\{n, \varphi(m)\}$ extra key switches are required; the number of external products nevertheless remains equal to n .

3 Enhanced Blind Rotation Algorithms

In this section, we present a Horner-like method for the blind rotation, which is a minor variation of [LMK⁺23, Algorithm 7]. Our reformulation primarily offers the advantage of providing a simpler basis for analyzing the results of Section 4. We also propose a new method derived from it—the *Traversal Windowed-Horner method*—which consistently outperforms [LMK⁺23]. These methods aim to reduce both the size of the keys and the number of automorphism applications.

3.1 Windowed-Horner Method

The idea of re-arranging the mask components $\tilde{a}_i \in (\mathbb{Z}/m\mathbb{Z})^\times$ has been extended in [LMK⁺23, Section 3], where the $\tilde{a}_i$'s are not only regrouped by values, but also ordered according to the group structure of $(\mathbb{Z}/m\mathbb{Z})^\times$. The immediate consequence is a reduction of the number of automorphism keys down to the number of cyclic components of $(\mathbb{Z}/m\mathbb{Z})^\times$.

As an illustration, following [LMK⁺23], we focus on the power-of-two conductor case.³ Let $m = 2N$ with $N = 2^\nu$, $\nu \geq 2$, so that $(\mathbb{Z}/m\mathbb{Z})^\times = \langle -1 \rangle \times \langle g \rangle$ using e.g., $g = 5$. Then, every element $\tilde{a}_i \in (\mathbb{Z}/m\mathbb{Z})^\times$ can be written as

$$\tilde{a}_i = (-1)^{\epsilon_i} \cdot g^{t_i} \pmod{m}, \quad \epsilon_i \in \{0, 1\}, \quad 0 \leq t_i < 2^{\nu-1}.$$

The high-level core idea of [LMK⁺23, Section 3.1] can be alternatively expressed as adapting the ordering of the mask components so as to ensure that the quotient $\tilde{a}_{\text{old}}/\tilde{a}_i$ in the telescoping method (Figure 2.1(b)) is always a small power of g . In order to formalize this, it is useful to introduce the sets

$$I_t^+ := \left\{ i \in \llbracket 1, n \rrbracket \mid \tilde{a}_i = g^t \pmod{2N} \right\} \quad (3.1)$$

and

$$I_t^- := \left\{ i \in \llbracket 1, n \rrbracket \mid \tilde{a}_i = -g^t \pmod{2N} \right\}. \quad (3.2)$$

Then the computation of $\langle \tilde{\mathbf{a}}, \mathbf{s} \rangle = \sum_{i=1}^n \tilde{a}_i s_i$ can be reordered as

$$\sum_{i=1}^n (-1)^{\epsilon_i} g^{t_i} \cdot s_i = \sum_{t=0}^{N/2-1} g^t \cdot \left(\sum_{i \in I_t^+} s_i \right) - \sum_{t=0}^{N/2-1} g^t \cdot \left(\sum_{i \in I_t^-} s_i \right), \quad (3.3)$$

which is naturally computed in a Horner-like fashion as

$$\begin{aligned} & \sum_{i \in I_0^+} s_i + g \left(\sum_{i \in I_1^+} s_i + \cdots + g \left(\sum_{i \in I_{N/2-1}^+} s_i \right. \right. \\ & \quad \left. \left. - g \left(\sum_{i \in I_0^-} s_i + g \left(\sum_{i \in I_1^-} s_i + \cdots + g \left(\sum_{i \in I_{N/2-1}^-} s_i \right) \right) \right) \right) \right) \right). \end{aligned} \quad (3.4)$$

This results in Algorithm 3.1, which is an almost (see Remark 3.1) equivalent rewriting of [LMK⁺23, Algorithm 7], reorganized so as to follow the work-flow of Figure 2.1(b). Thus, starting from $\tilde{a}_{\text{old}} = -1 = -g^{N/2} \pmod{2N}$, it iterates on non-empty sets $I_t^\pm$ so that for $i \in I_t^\pm$, $\tilde{a}_{\text{old}}/\tilde{a}_i$ is always the smallest possible power of g . Gaps, when some of the $I_t^\pm$ are empty, are filled using a jumping strategy defined by a window size w and automorphism keys

$$\begin{aligned} \mathbf{ak}^{\text{HORN}} := & \left\{ \mathbf{ak}^{\text{HORN}}[0] \leftarrow \mathbf{ksk}_{\tau_{-g}(\mathfrak{s}) \rightarrow \mathfrak{s}} \right\} \cup \\ & \left\{ \mathbf{ak}^{\text{HORN}}[u] \leftarrow \mathbf{ksk}_{\tau_{g^u}(\mathfrak{s}) \rightarrow \mathfrak{s}} \mid u \in \llbracket 1, w \rrbracket \right\}. \end{aligned} \quad (3.5)$$

Using $w = 1$ corresponds to repeated applications of τ_g , one per empty set $I_t^\pm$; a larger value for w decreases the number of calls to $\text{HomAut}()$ at the expense of larger key material. Practical experiments from [LMK⁺23, Figure 3] suggest a rather small optimal window value $w = 10$, further discussed in Section 5.2.

³ Although only the power-of-two conductor case is treated in [LMK⁺23], their core result readily applies to e.g., the simpler case of prime-conductor cyclotomic fields.

Algorithm 3.1: Blind rotation w/automorphisms — Windowed-Horner

Input: $\tilde{c} \leftarrow (\tilde{a}_1, \dots, \tilde{a}_n, \tilde{b}) \in (\mathbb{Z}/2N\mathbb{Z})^{n+1}$, $\tilde{a}_i \in (\mathbb{Z}/2N\mathbb{Z})^\times \cup \{0\}$; $v \in \mathcal{R}_q$
Data: bsk^{AUT} and ak^{HORN} as in [Eqns. \(2.3\)](#) and [\(3.5\)](#) for a window size $w \geq 1$
Output: $c \leftarrow \text{GLWE}_s(x^{-\tilde{\mu}} \cdot v) \in \mathcal{R}_q^{k+1}$ with $\tilde{\mu} = \tilde{b} - \sum_{i=1}^n \tilde{a}_i s_i$

1. $t_{\text{old}} \leftarrow 0$, $\text{ACC} \leftarrow (0, \dots, 0, x^{\tilde{b}} \cdot v(x^{-1}))$ /* i.e., $\tilde{a}_{\text{old}} = -g^{N/2} = -1$ */
2. **for** $t = N/2 - 1$ **down to** 0 **such that** $I_t^- \neq \emptyset$ **do** /* see [Equation \(3.2\)](#) */
3. $\delta \leftarrow (t_{\text{old}} - t) \bmod N/2$, $t_{\text{old}} \leftarrow t$
4. /* Homomorphically apply τ_{g^δ} using jumps of size at most w */
5. $q_\delta \leftarrow \lfloor \delta/w \rfloor$, $r_\delta \leftarrow \delta \bmod w$
6. **for** q_δ **times do** $\text{ACC} \leftarrow \text{HomAut}_{g^w}(\text{ACC}, \text{ak}^{\text{HORN}}[w])$
7. **if** $r_\delta \neq 0$ **then** $\text{ACC} \leftarrow \text{HomAut}_{g^{r_\delta}}(\text{ACC}, \text{ak}^{\text{HORN}}[r_\delta])$
8. /* Compute all external products for I_t^- */
9. **for** $i \in I_t^-$ **do**
10. $\text{ACC} \leftarrow \text{ACC} \otimes \text{bsk}^{\text{AUT}}[i]$
11. /* Apply τ_{-g} as in second line of [Equation \(3.4\)](#), see [Remark 3.1](#) */
12. $t_{\text{old}} \leftarrow (t_{\text{old}} - 1) \bmod N/2$, $\text{ACC} \leftarrow \text{HomAut}_{-g}(\text{ACC}, \text{ak}^{\text{HORN}}[0])$
13. /* Same loop as the first loop, but for (non-empty) sets I_t^+ ([Equation \(3.1\)](#)) */
14. **for** $t = N/2 - 1$ **down to** 0 **such that** $I_t^+ \neq \emptyset$ **do**
15. $\delta \leftarrow (t_{\text{old}} - t) \bmod N/2$, $t_{\text{old}} \leftarrow t$
16. $q_\delta \leftarrow \lfloor \delta/w \rfloor$, $r_\delta \leftarrow \delta \bmod w$
17. **for** q_δ **times do** $\text{ACC} \leftarrow \text{HomAut}_{g^w}(\text{ACC}, \text{ak}^{\text{HORN}}[w])$
18. **if** $r_\delta \neq 0$ **then** $\text{ACC} \leftarrow \text{HomAut}_{g^{r_\delta}}(\text{ACC}, \text{ak}^{\text{HORN}}[r_\delta])$
19. **for** $i \in I_t^+$ **do**
20. $\text{ACC} \leftarrow \text{ACC} \otimes \text{bsk}^{\text{AUT}}[i]$
21. $\delta \leftarrow t_{\text{old}}$, $q_\delta \leftarrow \lfloor \delta/w \rfloor$, $r_\delta \leftarrow \delta \bmod w$
22. **for** q_δ **times do** $\text{ACC} \leftarrow \text{HomAut}_{g^w}(\text{ACC}, \text{ak}^{\text{HORN}}[w])$
23. **if** $r_\delta \neq 0$ **then** $\text{ACC} \leftarrow \text{HomAut}_{g^{r_\delta}}(\text{ACC}, \text{ak}^{\text{HORN}}[r_\delta])$
24. **return** ACC

Remark 3.1. In [\[LMK⁺23\]](#), a “flush” homomorphically applying τ_{g^δ} is required before moving to the second loop, as shown by the condition “ $\lfloor \dots \rfloor$ or $\ell = 1$ ” in [\[LMK⁺23, Algorithm 3, Line 7\]](#). Our rewriting and proof ([Appendix B](#)) makes visible that τ_{-g} can be applied at any time after handling the last non-empty set I_t^- , whereas filling the gap with τ_{g^δ} can be deferred to the second loop, simply adjusting t_{old} as in [Line 9](#). This often saves one $\text{HomAut}()$ evaluation.

3.2 A New Traversal Windowed-Horner Method

Since n is typically small w.r.t. $2N$, many sets I_t^+ or I_t^- are empty, which implies that $I_t := I_t^- \cup I_t^+$ is often equal to either I_t^+ or I_t^- . This suggests a better strategy to enumerate $(\mathbb{Z}/2N\mathbb{Z})^\times$, where closing the gap between two non-empty sets I_t ’s can be directly combined with a sign change. The main effect is to reduce $\mathbb{E}[\delta]$,

Algorithm 3.2: BR w/automorphisms — Traversal Windowed-Horner

Input: $\tilde{c} \leftarrow (\tilde{a}_1, \dots, \tilde{a}_n, \tilde{b}) \in (\mathbb{Z}/2N\mathbb{Z})^{n+1}$, $\tilde{a}_i \in (\mathbb{Z}/2N\mathbb{Z})^\times \cup \{0\}$; $v \in \mathcal{R}_q$
Data: bsk^{AUT} and ak^{TRAV} as in [Eqns. \(2.3\)](#) and [\(3.6\)](#) for a window size $w' = w/2$
Output: $c \leftarrow \text{GLWE}_\delta(x^{-\tilde{\mu}} \cdot v) \in \mathcal{R}_q^{k+1}$ with $\tilde{\mu} = \tilde{b} - \sum_{i=1}^n \tilde{a}_i s_i$

1. $\epsilon_{\text{old}} \leftarrow +1$, $t_{\text{old}} \leftarrow N/2$, $\text{ACC} \leftarrow (0, \dots, 0, x^{-\tilde{b}} \cdot v(x))$
2. **for** $t = N/2 - 1$ **down to** 0 **do**
 3. **for** $\epsilon \in \{\epsilon_{\text{old}}, -\epsilon_{\text{old}}\}$ **such that** $I_t^\epsilon \neq \emptyset$ **do**
 4. */* Compute $\sigma \cdot g^\delta = \epsilon_{\text{old}} \cdot g^{t_{\text{old}}} / (\epsilon \cdot g^t)$, update tracking values */*
 $\delta \leftarrow t_{\text{old}} - t$, $t_{\text{old}} \leftarrow t$, $\sigma \leftarrow \epsilon_{\text{old}} / \epsilon$, $\epsilon_{\text{old}} \leftarrow \epsilon$
 5. */* Apply τ_u for $u = \sigma \cdot g^\delta$, see [Remark 3.2](#) */*
if $\delta = 0$ **then** $\text{ACC} \leftarrow \text{HomAut}_{-1}(\text{ACC}, \text{ak}^{\text{TRAV}}[0])$
 6. **else**
 7. Write $\delta = q_\delta \cdot w' + r_\delta$ with $r_\delta \in \llbracket 1, w' \rrbracket$ and $q_\delta \geq 0$
 8. **for** q_δ times **do** $\text{ACC} \leftarrow \text{HomAut}_{g^{w'}}(\text{ACC}, \text{ak}^{\text{TRAV}}[w'])$
 9. $\text{ACC} \leftarrow \text{HomAut}_{\sigma \cdot g^{r_\delta}}(\text{ACC}, \text{ak}^{\text{TRAV}}[\sigma \cdot r_\delta])$
 10. */* Compute all external products for I_t^ϵ */*
for $i \in I_t^\epsilon$ **do**
 11. $\text{ACC} \leftarrow \text{ACC} \otimes \text{bsk}^{\text{AUT}}[i]$
 12. */* Finally, apply τ_u for $u = \epsilon_{\text{old}} \cdot g^{t_{\text{old}}}$ */*
if $t_{\text{old}} = 0$ **and** $\epsilon_{\text{old}} = -1$ **then** $\text{ACC} \leftarrow \text{HomAut}_{-1}(\text{ACC}, \text{ak}^{\text{TRAV}}[0])$
 13. **else if** $t_{\text{old}} \neq 0$ **then**
 14. Write $\delta = t_{\text{old}} = q_\delta \cdot w' + r_\delta$ with $r_\delta \in \llbracket 1, w' \rrbracket$ and $q_\delta \geq 0$
 15. **for** q_δ times **do** $\text{ACC} \leftarrow \text{HomAut}_{g^{w'}}(\text{ACC}, \text{ak}^{\text{TRAV}}[w'])$
 16. $\text{ACC} \leftarrow \text{HomAut}_{\epsilon_{\text{old}} \cdot g^{r_\delta}}(\text{ACC}, \text{ak}^{\text{TRAV}}[\epsilon_{\text{old}} \cdot r_\delta])$
 17. **return** ACC

the average size of the gaps. As will be demonstrated, this enhances efficiency, especially for smaller window sizes.

Doing so yields [Algorithm 3.2](#), which exchanges the loops of [Algorithm 3.1](#) by including the sign change, whenever needed, directly inside a unique loop on t . The required automorphism keys are defined as

$$\text{ak}^{\text{TRAV}} := \left\{ \text{ak}^{\text{TRAV}}[0] \leftarrow \text{ksk}_{\tau_{-1}(\mathfrak{s}) \rightarrow \mathfrak{s}} \right\} \cup \left\{ \text{ak}^{\text{TRAV}}[\pm u] \leftarrow \text{ksk}_{\tau_{\pm g^u}(\mathfrak{s}) \rightarrow \mathfrak{s}} \mid u \in \llbracket 1, w' \rrbracket \right\}. \quad (3.6)$$

Remark 3.2. We chose to always combine a possible sign change with the application of $\tau_{g^{r_\delta}}$. When $\delta \neq 0$, it implies to modify the definition of q_δ to $\lfloor \frac{\delta-1}{w'} \rfloor$ and $r_\delta = \delta - q_\delta \cdot w'$ to ensure $r_\delta \in \llbracket 1, w' \rrbracket$ so that $\text{ak}^{\text{TRAV}}[\sigma \cdot r_\delta]$ exists. Further, the case $\delta = 0$ can only occur when $\epsilon = -\epsilon_{\text{old}}$, i.e., $\sigma = -1$ inside the loop.

Proposition 3.3. *Algorithm 3.2 is correct.*

Proof. For $t \in \llbracket 0, N/2 \rrbracket$, define recursively $\tilde{q}_t \in \mathcal{R}_q$ by $\tilde{q}_{N/2} = v \cdot x^{-\tilde{b}}$ and

$$\tilde{q}_t = \tilde{q}_{t+1} \cdot x^{g^t \cdot (\sum_{i \in I_t^+} s_i - \sum_{i \in I_t^-} s_i)}.$$

We inductively show that after each iteration, ACC contains a GLWE encryption of $\tau_{\epsilon_{\text{old}} \cdot g^{-t_{\text{old}}}}(\tilde{q}_t)$.

First, ACC is initialized to a (trivial) GLWE encryption of $x^{-\tilde{b}} \cdot v(x)$, which is indeed equal to $\tau_{+g^{N/2}}(\tilde{q}_{N/2})$. Inside the loop, the induction hypothesis is preserved as long as both I_t^+ and I_t^- are empty, since in this case $\tilde{q}_t = \tilde{q}_{t_{\text{old}}}$. Assume now, for $N/2 > t \geq 0$, that ACC contains a GLWE encryption of $\tau_{\epsilon_{\text{old}} \cdot g^{-t_{\text{old}}}}(\tilde{q}_{t+1})$, and exactly one I_t^ϵ is non-empty for $\epsilon \in \{\epsilon_{\text{old}}, -\epsilon_{\text{old}}\}$. Then, $\tau_{\sigma \cdot g^\delta}$ ($\delta \geq 1$) is homomorphically applied to ACC to obtain a GLWE encryption of $\tau_{\epsilon g^{-t}}(\tilde{q}_{t+1})$, and the following external products finally yield as expected a GLWE encryption of

$$\tau_{\epsilon g^{-t}}(\tilde{q}_{t+1}) \cdot x^{\sum_{i \in I_t^\epsilon} s_i} = \tau_{\epsilon g^{-t}}(\tilde{q}_{t+1} \cdot x^{g^t \sum_{i \in I_t^\epsilon} \epsilon s_i}) = \tau_{\epsilon g^{-t}}(\tilde{q}_t).$$

When both $I_t^\pm$ are non-empty, the previous reasoning is first applied on $I_t^{\epsilon_{\text{old}}}$; then for $\epsilon = -\epsilon_{\text{old}}$, we have $\delta = 0$ so τ_{-1} is applied to ACC and external products corresponding to indices in I_t^ϵ finally yield a GLWE encryption of

$$\begin{aligned} \tau_{-\epsilon_{\text{old}} \cdot g^{-t}}(\tilde{q}_{t+1} \cdot x^{\epsilon_{\text{old}} \cdot g^t \sum_{i \in I_t^{\epsilon_{\text{old}}}} s_i}) \cdot x^{\sum_{i \in I_t^\epsilon} s_i} \\ = \tau_{\epsilon g^{-t}}(\tilde{q}_{t+1} \cdot x^{g^t (\sum_{i \in I_t^\epsilon} \epsilon s_i + \sum_{i \in I_t^{-\epsilon}} (-\epsilon) s_i)}) = \tau_{\epsilon g^{-t}}(\tilde{q}_t), \end{aligned}$$

which again is the induction hypothesis with [Line 4](#) adjustments on ϵ_{old} and t_{old} . Hence, after the loop, ACC contains a GLWE encryption of $\tau_{\epsilon_{\text{old}} \cdot g^{-t_{\text{old}}}}(\tilde{q}_0)$, implying the result since the last lines of the algorithm apply $\tau_{\epsilon_{\text{old}} \cdot g^{t_{\text{old}}}}$. $\square$

For completeness, we provide in [Tables 3.1\(a\)](#) and [3.1\(b\)](#) a numerical comparison of the expected number of automorphism key switches in [Algorithm 3.1](#) (similar to [\[LMK⁺23, Algorithm 7\]](#)) vs. [Algorithm 3.2](#) under two different parameter sets. The first set is taken from [\[LMK⁺23, Table 2\]](#) and the second set is `PARAM_MESSAGE_2_CARRY_2_KS_PBS_GAUSSIAN_2M64`⁴ from `TFHE-rs` [\[Zam22\]](#), with parameters $(n, N, k) = (458, 1024, 1)$ and $(n, N, k) = (834, 2048, 1)$, respectively. For an easier comparison, we assume $w = 2w'$ so that in both cases the key material includes $w + 1$ automorphism keys. These tables show that, although both methods roughly converge to the same optimum, the new traversal method always shows superior performance due to its ability to combine smaller jumps on average with sign changes; in particular, for small values of $w = 2w'$.

4 Automorphism-Parametrized Techniques

We propose a generalized external product that seamlessly incorporates a homomorphic automorphism evaluation on the input ciphertext. Specifically, by

⁴ Git commit [400ce27beb5bea8fdc68826ad437099ec62680d0](https://github.com/TFHE-rs/TFHE-rs/commit/400ce27beb5bea8fdc68826ad437099ec62680d0), Sept. 25, 2024.

Table 3.1: Measured expected number of automorphism key switches for [Algorithms 3.1](#) and [3.2](#) under two different parameters sets.

(a) For $n = 458$ and $N = 1024$.				(b) For $n = 834$ and $N = 2048$.			
$w = 2w'$	Alg. 3.1	Alg. 3.2	Ratio (%)	$w = 2w'$	Alg. 3.1	Alg. 3.2	Ratio (%)
$w = 2$	625	578	92.5	$w = 2$	1232	1139	92.5
$w = 4$	445	431	96.7	$w = 4$	857	827	96.5
$w = 6$	396	390	98.4	$w = 6$	755	743	98.4
$w = 8$	380	377	99.3	$w = 8$	721	715	99.2
$w = 10$	373	371	99.5	$w = 10$	702	698	99.4
$w = 12$	372	370	99.6	$w = 12$	698	696	99.7
$w = 14$	372	370	99.7	$w = 14$	692	691	99.8

using a *modified* $\text{GLWE}^\circledast$ -like ciphertext, we show how the key switch associated with the homomorphic automorphism evaluation can be absorbed within the external product. In essence, our new operator enables in a single step and at the cost of a single external product, the execution of a homomorphic automorphism evaluation—including its key switch—followed by an external product.

4.1 Automorphism-Parametrized External Product

We generalize the extended-GLWE ($\text{GLWE}^\circledast$, aka GGSW) ciphertexts, which are used for computing external products, to embed information about the image of the key under a given automorphism ψ ; i.e., $\psi = \tau_u$ for a fixed $u \in (\mathbb{Z}/m\mathbb{Z})^\times$.

Definition 4.1. An Automorphism-Extended-GLWE ciphertext relatively to automorphism ψ and to gadget decomposition $\nabla = \nabla_{\ell_1, \ell_2}$ of a plaintext $\bar{\mu} \in \mathcal{R}_q$ under key $\mathfrak{s} \in \mathcal{R}_q^k$ is denoted by $\text{GLWE}_{\mathfrak{s}}^{\circledast, \psi}(\bar{\mu})$ and defined as

$$\left\{ \text{GLWE}_{\mathfrak{s}}^{\nabla_{\ell_1}}(-\psi(\mathfrak{s}_1) \cdot \bar{\mu}), \dots, \text{GLWE}_{\mathfrak{s}}^{\nabla_{\ell_1}}(-\psi(\mathfrak{s}_k) \cdot \bar{\mu}), \text{GLWE}_{\mathfrak{s}}^{\nabla_{\ell_2}}(\bar{\mu}) \right\}.$$

In particular, for a given decomposition ∇ , it holds that $\text{GLWE}_{\mathfrak{s}}^{\circledast, \text{id}} = \text{GLWE}_{\mathfrak{s}}^\circledast$.

Such ciphertexts $\text{GLWE}_{\mathfrak{s}}^{\circledast, \psi}$ enable the combination of a homomorphic evaluation of ψ on a GLWE input with an external product, *without* requiring an intermediate key switch, as demonstrated below.

Definition 4.2 (Automorphism-parametrized external product). The automorphism-parametrized external product, relatively to the automorphism ψ and gadget decomposition $\nabla = \nabla_{\ell_1, \ell_2}$, is denoted by $\otimes_\psi$ and defined as

$$\otimes_\psi : \text{GLWE}_{\mathfrak{s}}(\mu) \times \text{GLWE}_{\mathfrak{s}}^{\circledast, \psi}(\bar{\mu}) \longrightarrow \text{GLWE}_{\mathfrak{s}}(\psi(\mu) \cdot \bar{\mu}),$$

where, for $\text{GLWE}_{\mathfrak{s}}(\mu) = (\alpha_1, \dots, \alpha_k, \mathfrak{t})$, the result is computed as

$$\left\langle \nabla_{\ell_2} \psi(\mathfrak{t}), \text{GLWE}_{\mathfrak{s}}^{\nabla_{\ell_2}}(\bar{\mu}) \right\rangle + \sum_{j=1}^k \left\langle \nabla_{\ell_1} \psi(\alpha_j), \text{GLWE}_{\mathfrak{s}}^{\nabla_{\ell_1}}(-\psi(\mathfrak{s}_k) \cdot \bar{\mu}) \right\rangle.$$

In particular, for a given decomposition ∇ , $\otimes_{\text{id}}$ coincides with $\otimes$.

The noise associated to this new operation is given by [Proposition 4.3](#), which essentially highlights a gain due to the removal of the key switch. It depends on a constant C_∞ which is set to 1 when m is a power of two. In the general case where m is not a power of two, $C_\infty > 1$ corresponds to the expansion factor of $\Phi_m(x)$, e.g., $C_\infty = 2$ when m is a prime $p > 2$ [\[MKMS24\]](#) or $m = 3^a$ [\[JW22\]](#), and $C_\infty = 4$ when $m = 2^b 3^a$ for $b \geq 2$ [\[JW22\]](#). Notably, for any automorphism ψ and all $w \in \mathcal{R}_q$, it holds that $\|\psi(w)\|_\infty \leq C_\infty \cdot \|w\|_\infty$.

Proposition 4.3. *Let $\nabla = \nabla_{\ell_1, \ell_2}$ be a gadget decomposition of quality $\beta_\nabla = (\beta_1, \beta_2)$ and precision $\varepsilon_\nabla = (\varepsilon_1, \varepsilon_2)$, whose output values are uniform and centered around 0. Let e_{in} and $\bar{\mathbf{e}}$ represent the error associated with valid samples $\text{GLWE}_\mathfrak{s}(\mu)$ and $\text{GLWE}_\mathfrak{s}^{\otimes, \psi}(\bar{\mu})$, respectively. Then $\text{GLWE}_\mathfrak{s}(\mu) \otimes_\psi \text{GLWE}_\mathfrak{s}^{\otimes, \psi}(\bar{\mu})$ is a sample of $\text{GLWE}_\mathfrak{s}(\psi(\mu) \cdot \bar{\mu})$ whose error has variance*

$$\sigma_{\otimes_\psi}^2 \leq C_\infty \cdot \left(\|\bar{\mu}\|_2^2 \cdot \sigma_{\text{in}}^2 + N \left(\ell_2 \frac{\beta_2^2}{12} + k \ell_1 \frac{\beta_1^2}{12} \right) \cdot \sigma_\nabla^2 + \|\bar{\mu}\|_2^2 \left(\frac{\varepsilon_2^2}{12} + k N \frac{\varepsilon_1^2}{12} \cdot \mathbb{E}[\mathfrak{s}_{j,i}^2] \right) \right).$$

Proof. Let $\mathbf{L}_0 \in \mathcal{R}_q^{k \times \ell_2}$ s.t. $\text{GLWE}_\mathfrak{s}^{\nabla_{\ell_2}}(\bar{\mu}) = (\mathbf{L}_0, \mathfrak{t}_0)$, i.e., the ℓ_2 columns of $\mathbf{L}_0$ are the respective masks of each $\text{GLWE}_\mathfrak{s}(\mathfrak{g}_{2,i}\bar{\mu})$, and $\mathfrak{t}_0 = \mathfrak{s} \cdot \mathbf{L}_0 + \bar{\mu} \cdot \mathfrak{g}_2 + \bar{\mathbf{e}}_0$. Similarly, let for $j \in \llbracket 1, k \rrbracket$, $\mathbf{L}_j \in \mathcal{R}_q^{k \times \ell_1}$ s.t. $\text{GLWE}_\mathfrak{s}^{\nabla_{\ell_1}}(-\psi(\mathfrak{s}_j) \cdot \bar{\mu}) = (\mathbf{L}_j, \mathfrak{t}_j)$, where $\mathfrak{t}_j = \mathfrak{s} \cdot \mathbf{L}_j - \psi(\mathfrak{s}_j) \cdot \bar{\mu} \cdot \mathfrak{g}_1 + \bar{\mathbf{e}}_j$. For $\text{GLWE}_\mathfrak{s}(\mu) = (\mathfrak{a}, \mathfrak{t})$, the resulting ciphertext of the above-defined operation is $(\mathfrak{A}, \mathfrak{B})$ with

$$\begin{cases} \mathfrak{A} = \nabla_{\ell_2} \psi(\mathfrak{t}) \cdot \mathbf{L}_0^\top + \sum_{1 \leq j \leq k} \nabla_{\ell_1} \psi(\mathfrak{a}_j) \cdot \mathbf{L}_j^\top & \in \mathcal{R}_q^k \\ \mathfrak{B} = \langle \nabla_{\ell_2} \psi(\mathfrak{t}), \mathfrak{t}_0 \rangle + \sum_{1 \leq j \leq k} \langle \nabla_{\ell_1} \psi(\mathfrak{a}_j), \mathfrak{t}_j \rangle & \in \mathcal{R}_q \end{cases}.$$

Rearranging terms after expanding the definition of the $\mathfrak{t}_j$'s yields

$$\begin{aligned} \mathfrak{B} = \langle \mathfrak{A}, \mathfrak{s} \rangle + \bar{\mu} \cdot \left(\langle \nabla_{\ell_2} \psi(\mathfrak{t}), \mathfrak{g}_2 \rangle - \sum_{1 \leq j \leq k} \psi(\mathfrak{s}_j) \cdot \langle \nabla_{\ell_1} \psi(\mathfrak{a}_j), \mathfrak{g}_1 \rangle \right) \\ + \left(\langle \nabla_{\ell_2} \psi(\mathfrak{t}), \bar{\mathbf{e}}_0 \rangle + \sum_{1 \leq j \leq k} \langle \nabla_{\ell_1} \psi(\mathfrak{a}_j), \bar{\mathbf{e}}_j \rangle \right), \end{aligned}$$

hence the exact expression of the error term $\mathfrak{E} = \mathfrak{B} - \langle \mathfrak{A}, \mathfrak{s} \rangle - \psi(\mu) \cdot \bar{\mu}$ is given by

$$\begin{aligned} \mathfrak{E} = \bar{\mu} \cdot \psi(e_{\text{in}}) + \left(\langle \nabla_{\ell_2} \psi(\mathfrak{t}), \bar{\mathbf{e}}_0 \rangle + \sum_{1 \leq j \leq k} \langle \nabla_{\ell_1} \psi(\mathfrak{a}_j), \bar{\mathbf{e}}_j \rangle \right) \\ + \bar{\mu} \cdot \left(e_{\nabla_{\ell_2}}(\psi(\mathfrak{t})) - \sum_{1 \leq j \leq k} \psi(\mathfrak{s}_j) \cdot e_{\nabla_{\ell_1}}(\psi(\mathfrak{a}_j)) \right), \end{aligned}$$

where $e_{\nabla_{\ell_u}}(w) := \langle \nabla_{\ell_u} w, \mathfrak{g}_u \rangle - w$ for $u \in \{1, 2\}$ and any $w \in \mathcal{R}_q$.

Polynomials output by the decomposition are supposed uniform and centered around 0, independently of ψ , and uncorrelated with error polynomials $\bar{\mathbf{e}}_j$. Thus, the variance of e.g., $\langle \nabla_{\ell_1} \psi(\mathfrak{a}_j), \bar{\mathbf{e}}_j \rangle$ is upper-bounded by $\ell_1 \cdot C_\infty N \cdot \frac{\beta_1^2}{12} \sigma_\nabla^2$. Similarly, the variance of the coefficients of $\bar{\mu} \cdot \psi(e_{\text{in}})$ is given by $\|\bar{\mu}\|_2^2 \cdot C_\infty \sigma_{\text{in}}^2$. By hypothesis, the coefficients of $e_{\nabla_{\ell_u}}(w)$ have variance upper-bounded by $\frac{\varepsilon_u^2}{12}$. Specifically, for the terms of the form $\bar{\mu} \cdot \psi(\mathfrak{s}_j) \cdot e_{\nabla_{\ell_1}}(\psi(\mathfrak{a}_j))$, the final variance is $\|\bar{\mu}\|_2^2 \cdot C_\infty (\sigma_{\mathfrak{s}_j}^2 + \mathbb{E}[\mathfrak{s}_j^2]) \cdot N \frac{\varepsilon_1^2}{12}$. Bringing all of these together yields the result. $\square$

4.2 Reducing Automorphism Key Switches in Blind Rotation

We now show how to leverage our new automorphism-parametrized external product to reduce the number of key switches in the Traversal Windowed-Horner method. In [Algorithm 3.2](#), the transition from $I_{t_{\text{old}}}^{\epsilon_{\text{old}}}$ to I_t^ϵ involves a homomorphic automorphism evaluation, which includes at least one automorphism key switch. Roughly speaking, the new operation combines this homomorphic automorphism evaluation with the first external product in I_t^ϵ . In effect, this narrows the distance between two consecutive sets by eliminating the associated key switch.

For each gap addressed in this manner, e.g., corresponding to an automorphism $\psi = \tau_{\pm g^\delta}$, additional $\text{GLWE}^{\otimes, \psi}$ -ciphertexts are required. This creates a trade-off between the size of the keys (i.e., the size of the set of admissible gaps) and reduced performance and increased noise growth (i.e., more key switches). However, as shown in [Section 5](#), around 63% of the gaps in [Algorithm 3.2](#) are covered by $\{\tau_{-1}, \tau_{\pm g}\}$ and around 84% by $\{\tau_{-1}, \tau_{\pm g}, \tau_{\pm g^2}\}$.

Formal description Let $\mathcal{S}$ denote the set of admissible automorphisms, and assume that $\text{id} \in \mathcal{S}$.⁵ For convenience, a dual set $\mathcal{S}_* := \{(\delta, \epsilon) \mid \epsilon \in \{\pm 1\}, \tau_{\epsilon \cdot g^\delta} \in \mathcal{S}\}$ is defined, which encodes $\mathcal{S}$ in $(\mathbb{Z}/2N\mathbb{Z})^\times$. In particular, by the assumption on $\mathcal{S}$, it follows that $(0, 1) \in \mathcal{S}_*$. The associated keys are defined by

$$\text{bsk}^{\mathcal{S}\text{-AUT}} := \left\{ \text{bsk}_{\psi}^{\mathcal{S}\text{-AUT}}[i] \leftarrow \text{GLWE}_{\mathfrak{d}}^{\otimes, \psi}(x^{s_i}) \mid i \in \llbracket 1, n \rrbracket, \psi \in \mathcal{S} \right\}, \quad (4.1)$$

and

$$\begin{aligned} \text{ak}^{\mathcal{S}\text{-AUT}} := & \left\{ \text{ak}^{\mathcal{S}\text{-AUT}}[0] \leftarrow \text{ksk}_{\tau_{-1}(\mathfrak{d}) \rightarrow \mathfrak{d}} \right\} \cup \\ & \left\{ \text{ak}^{\mathcal{S}\text{-AUT}}[\pm u] \leftarrow \text{ksk}_{\tau_{\pm g^u}(\mathfrak{d}) \rightarrow \mathfrak{d}} \mid u \in \llbracket 1, w'' \rrbracket \right\}. \end{aligned} \quad (4.2)$$

For a fixed $i \in \llbracket 1, n \rrbracket$, it is important to note that $\text{bsk}_{\psi}^{\mathcal{S}\text{-AUT}}[i]$, $\psi \in \mathcal{S}$, all share the common term $\text{GLWE}_{\mathfrak{d}}^{\nabla_{\ell_2}}(x^{s_i})$. Although this term is repeated for notational clarity, it implies that the size of each $\text{bsk}^{\mathcal{S}\text{-AUT}}[i]$ is $(\#\mathcal{S} \cdot k + 1)$ Gadget-GLWE ciphertexts, rather than $\#\mathcal{S} \cdot (k + 1)$ Gadget-GLWE ciphertexts.

The new jumping strategy is formalized in [Algorithm 4.1](#). For a gap $u = \sigma \cdot g^\delta$, the approach involves finding the closest pair $(\delta_*, \epsilon_*) \in \mathcal{S}_*$, going forward, and thereafter decomposing the automorphism τ_u as $\psi \circ \tau_v$, where $\psi = \tau_{\epsilon_* \cdot g^{\delta_*}} \in \mathcal{S}$ and $v = u \cdot (\epsilon_* \cdot g^{\delta_*})^{-1}$. When the component τ_v is non-trivial, it is homomorphically applied to ACC with the windowed method as in [Algorithm 3.2](#), which requires at least one automorphism key switch, whereas $\psi \in \mathcal{S}$ is applied as part of the new automorphism-parametrized external product $\otimes_\psi$.

Remark 4.4. As with any other automorphism-based blind rotation, the first automorphism evaluation on ACC is entirely free. Therefore, knowing the first (t_0, ϵ_0) in the loop s.t. $I_{t_0}^{\epsilon_0} \neq \emptyset$, we can modify the initialization step by directly setting $\epsilon_{\text{old}} = \epsilon_0$, $t_{\text{old}} = t_0$ and $\text{ACC} = (0, \dots, 0, x^{-u^b} \cdot v(x^u))$ for $u = \epsilon_0 \cdot g^{-t_0}$.

⁵ While not strictly necessary, including id in $\mathcal{S}$ always yields similar or better trade-offs and simplifies both the presentation and the description of automorphism keys.

Algorithm 4.1: Blind Rotation: $\mathcal{S}$ -parametrized method

Input: $\tilde{c} \leftarrow (\tilde{a}_1, \dots, \tilde{a}_n, \tilde{b}) \in (\mathbb{Z}/2N\mathbb{Z})^{n+1}$, $\tilde{a}_i \in (\mathbb{Z}/2N\mathbb{Z})^\times \cup \{0\}$; $v \in \mathcal{R}_q$
Data: $\text{bsk}^{\mathcal{S}\text{-AUT}}$ and $\text{ak}^{\mathcal{S}\text{-AUT}}$ as defined in Equations (4.1) and (4.2) for a window size w'' and a set $\mathcal{S}$ of admissible automorphisms
Output: $c \leftarrow \text{GLWE}_\delta(x^{-\tilde{\mu}} \cdot v) \in \mathcal{R}_q^{k+1}$ with $\tilde{\mu} = \tilde{b} - \sum_{i=1}^n \tilde{a}_i s_i$

1. $\epsilon_{\text{old}} \leftarrow +1$, $t_{\text{old}} \leftarrow N/2$, $\text{ACC} \leftarrow (0, \dots, 0, x^{-\tilde{b}} \cdot v(x))$ /* see Remark 4.4 */
2. **for** $t = N/2 - 1$ **down to** 0 **such that** $I_t^+ \cup I_t^- \neq \emptyset$ **do**
3. $\epsilon_{\text{first}} \leftarrow \epsilon_{\text{old}}$ **if** $(t_{\text{old}} - t, +1) \in \mathcal{S}_*$ **else** $-\epsilon_{\text{old}}$ /* see Remark 4.5 */
4. **for** $\epsilon \in \{\epsilon_{\text{first}}, -\epsilon_{\text{first}}\}$ **such that** $I_t^\epsilon \neq \emptyset$ **do**
5. /* Compute $\sigma \cdot g^\delta = \epsilon_{\text{old}} \cdot g^{t_{\text{old}}} / (\epsilon \cdot g^t)$, update tracking values */
6. $\delta \leftarrow t_{\text{old}} - t$, $t_{\text{old}} \leftarrow t$, $\sigma \leftarrow \epsilon_{\text{old}} / \epsilon$, $\epsilon_{\text{old}} \leftarrow \epsilon$
7. /* Jumping strategy: find $(\delta_*, \epsilon_*) \in \mathcal{S}_*$ alphabetically closest to (δ, σ) */
8. $\delta_* \leftarrow \max\{(\delta_*, \cdot) \in \mathcal{S}_* \mid \delta_* \leq \delta\}$, $\epsilon_* \leftarrow \sigma$ **if** $(\delta_*, \sigma) \in \mathcal{S}_*$ **else** $-\sigma$
9. /* Jumping strategy: apply τ_v for $v = \sigma \cdot g^\delta / (\epsilon_* \cdot g^{\delta_*})$ */
10. **if** $(\delta - \delta_*, \sigma / \epsilon_*) = (0, -1)$ **then** $\text{ACC} \leftarrow \text{HomAut}_{-1}(\text{ACC}, \text{ak}^{\mathcal{S}\text{-AUT}}[0])$
11. **else if** $(\delta - \delta_*) \neq 0$ **then**
12. Write $\delta - \delta_* = q_\delta \cdot w'' + r_\delta$ with $r_\delta \in \llbracket 1, w'' \rrbracket$ and $q_\delta \geq 0$
13. **for** q_δ times **do** $\text{ACC} \leftarrow \text{HomAut}_{g^{w''}}(\text{ACC}, \text{ak}^{\mathcal{S}\text{-AUT}}[w''])$
14. $\text{ACC} \leftarrow \text{HomAut}_{\sigma / \epsilon_* \cdot g^{r_\delta}}(\text{ACC}, \text{ak}^{\mathcal{S}\text{-AUT}}[\sigma / \epsilon_* \cdot r_\delta])$
15. /* Jumping strategy: first external product parametrized by $\psi = \tau_{\epsilon_* \cdot g^{\delta_*}}$ */
16. $\text{ACC} \leftarrow \text{ACC} \otimes_\psi \text{bsk}_\psi^{\mathcal{S}\text{-AUT}}[I_t^\epsilon[0]]$
17. /* Compute all remaining external products for I_t^ϵ */
18. **for** $i \in I_t^\epsilon \setminus \{I_t^\epsilon[0]\}$ **do**
19. $\text{ACC} \leftarrow \text{ACC} \otimes \text{bsk}_{\text{id}}^{\mathcal{S}\text{-AUT}}[i]$
20. /* Finally, apply τ_u for $u = \epsilon_{\text{old}} \cdot g^{t_{\text{old}}}$ (see Remark 4.6) */
21. **if** $\epsilon_{\text{old}} = -1$ **then** $\text{ACC} \leftarrow \text{HomAut}_{-1}(\text{ACC}, \text{ak}^{\mathcal{S}\text{-AUT}}[0])$
22. **if** $t_{\text{old}} \neq 0$ **then**
23. Write $t_{\text{old}} = q_\delta \cdot w'' + r_\delta$ with $r_\delta \in \llbracket 1, w'' \rrbracket$ and $q_\delta \geq 0$
24. **for** q_δ times **do** $\text{ACC} \leftarrow \text{HomAut}_{g^{w''}}(\text{ACC}, \text{ak}^{\mathcal{S}\text{-AUT}}[w''])$
25. $\text{ACC} \leftarrow \text{HomAut}_{g^{r_\delta}}(\text{ACC}, \text{ak}^{\mathcal{S}\text{-AUT}}[r_\delta])$
26. **return** ACC

Remark 4.5. In the specific case where both $I_t^\pm$ are non-empty, $\delta_* = t_{\text{old}} - t$, and the set $\mathcal{S}$ is not symmetric, i.e., it contains τ_u for some $\pm u = g^{t_{\text{old}} - t}$ but does not necessarily include τ_{-u} , it is preferable to first consider the sign ϵ_{first} that will lead to the automorphism in $\mathcal{S}$, which is not always ϵ_{old} , as done in Line 3.

Remark 4.6. It is also worth noting that, when $\mathcal{S} \setminus \{\tau_{\pm 1}\}$ is symmetric, then during the main loop only non-negative indices of $\text{ak}^{\mathcal{S}\text{-AUT}}$ are necessary. Consequently, we modified the final steps of Algorithm 4.1, in particular Line 15, in order to ensure they also only require $\text{ak}^{\mathcal{S}\text{-AUT}}[0] \cup \text{ak}^{\mathcal{S}\text{-AUT}}[1 \dots w'']$. Thus, in many cases, $\text{ak}^{\mathcal{S}\text{-AUT}}$ can be made twice shorter than indicated in Equation (4.2).

Proposition 4.7. Algorithm 4.1 is correct.

Proof. The iteration invariant is the same as in [Algorithm 3.2](#), i.e., after iteration t , ACC contains a GLWE encryption of $\tau_{\epsilon_{\text{old}} \cdot g^{-t_{\text{old}}}}(\tilde{q}_t)$ under key $\mathfrak{z}$. The output is $\text{HomAut}_u(\text{ACC})$ for $u = \epsilon_{\text{old}} \cdot g_{\text{old}}^t$, which yields $\text{GLWE}_{\mathfrak{z}}(\mathfrak{v} \cdot x^{-\tilde{b} + \langle \tilde{\mathbf{a}}, \mathbf{s} \rangle})$ by induction. Note that the modified inner loop initialization (see [Remark 4.5](#)) only modifies adaptively its order, but has no impact on correctness. $\square$

Noise analysis The noise growth of [Algorithm 4.1](#) is directly linked to the number of external products (whether automorphism-parametrized or not), which is always n , and the number $\kappa(w'')$ of remaining automorphism key switches, which gets smaller as $\sharp \mathcal{S}$ grows.

Proposition 4.8. *Let $\kappa(w'')$ be the number of automorphism key switches required by [Algorithm 4.1](#). Then, using the same notations and gadget decompositions hypotheses as in [Propositions 4.3](#) and [2.2](#), the error term of the output of [Algorithm 4.1](#) has variance*

$$\sigma_{\mathcal{S}\text{-AUT}}^2 \leq n \cdot \mathfrak{s}_{\otimes \mathcal{S}}^2 + \kappa(w'') \cdot \mathfrak{s}_{\text{aut}}^2 ,$$

$$\text{where } \begin{cases} \mathfrak{s}_{\otimes \mathcal{S}}^2 \leq C_{\infty} \left(N \left(\ell_2 \frac{\beta_2^2}{12} + k \ell_1 \frac{\beta_1^2}{12} \right) \cdot \sigma_{\nabla}^2 + \left(\frac{\epsilon_2^2}{12} + k N \frac{\epsilon_1^2}{12} \cdot \mathbb{E}[\mathfrak{z}_{j,i}^2] \right) \right) \\ \mathfrak{s}_{\text{aut}}^2 \leq C_{\infty} \left(N \left(k \ell_{\text{ks}} \frac{\beta_{\text{ks}}^2}{12} \right) \cdot \sigma_{\text{ks}}^2 + k N \left(\mathbb{E}[\mathfrak{z}_{j,i}^2] \cdot \frac{\epsilon_{\text{ks}}^2}{12} \right) \right) . \end{cases}$$

Proof. We first prove the result in the (unlikely) case $\kappa(w'') = 0$, i.e., when the algorithm consists of a series of (automorphism-parametrized or not) external products $\otimes_{\psi_t}$, $t \in \llbracket 1, n \rrbracket$, where $\psi_t = \tau_{u_t} \in \mathcal{S}$ and $\psi_n \circ \dots \circ \psi_1 = \text{id}$. Without any loss of generality, we assume that the LWE key indexes have been reordered so that the t -th operation $\otimes_{\psi_t}$ involves $\text{bsk}_{\psi_t}^{\mathcal{S}\text{-AUT}}[t] = \text{GLWE}_{\mathfrak{z}}^{\otimes, \psi_t}(x^{s_t})$.

We proceed by expressing directly the final error term.⁶ Let $\mathfrak{E}_t$ be the error term of $\text{ACC} = (a_1, \dots, a_k, \mathfrak{b})$ after $\otimes_{\psi_t}$. Using the same notations as in the proof of [Proposition 4.3](#), $\mathfrak{E}_t = x^{s_t} \cdot \psi_t(\mathfrak{E}_{t-1}) + (E_{\otimes}^{(t)} + x^{s_t} \cdot E_{\nabla, \psi_t}^{(t)})$, where

$$\begin{cases} E_{\otimes}^{(t)} = \langle \nabla_{\ell_2} \psi_t(\mathfrak{b}), \bar{\mathfrak{e}}_0^{(t)} \rangle + \sum_{1 \leq j \leq k} \langle \nabla_{\ell_1} \psi_t(a_j), \bar{\mathfrak{e}}_j^{(t, \psi_t)} \rangle \\ E_{\nabla, \psi_t}^{(t)} = e_{\nabla_{\ell_2}}(\psi_t(\mathfrak{b})) - \sum_{1 \leq j \leq k} \psi_t(\mathfrak{z}_j) \cdot e_{\nabla_{\ell_1}}(\psi_t(a_j)) , \end{cases}$$

with $e_{\nabla_{\ell_u}}(w) := \langle \nabla_{\ell_u} w, \mathbf{g}_u \rangle - w$ for $u \in \{1, 2\}$ and any $w \in \mathcal{R}_q$. A simple induction then yields, from $\mathfrak{E}_0 = 0$ and $\mathfrak{E}_1 = E_{\otimes}^{(1)}$, i.e., $E_{\nabla, \psi_1}^{(1)} = 0$,

$$\mathfrak{E}_n = \sum_{t=1}^n \left(\prod_{a=t+1}^n x^{s_a u_{a+1} \dots u_n} \right) \cdot \tau_{u_n \dots u_{t+1}} \left(E_{\otimes}^{(t)} + x^{s_t} \cdot E_{\nabla, \psi_t}^{(t)} \right) .$$

We continue by looking at the case of one single automorphism key switch. For a given $t \in \llbracket 1, n \rrbracket$, assume $\psi_t \notin \mathcal{S}$ can be decomposed as $\psi_t^* \circ \tau_{v_t}$ where $\psi_t^* \in \mathcal{S}$

⁶ Whenever $C_{\infty} > 1$, applying n times [Proposition 4.3](#) would result in an artificially large factor C_{∞}^n in the upper bound. Contrary to the claim in [\[MKMS24, Page 11\]](#), applying an automorphism alone does, in fact, affect the error by a factor of C_{∞} .

and $\text{ak}^{\text{S-AUT}}$ contains a key corresponding to $\tau_{v_t} \neq \text{id}$. From [Proposition 2.2](#), the noise term after $\text{HomAut}_{\tau_{v_t}}$ is then given by $\tau_{v_t}(\mathcal{E}_{t-1}) + (E_{\text{ks}}^{(t)} + E_{\nabla_{\text{ks}, \tau_{v_t}}}^{(t)})$, where

$$\begin{cases} E_{\text{ks}}^{(t)} = \sum_{1 \leq j \leq k} \langle \nabla_{\ell_{\text{ks}}} \tau_{v_t}(a_j), \bar{\mathbf{e}}_{\text{ks}, j}^{(v_t)} \rangle \\ E_{\nabla_{\text{ks}, \tau_{v_t}}}^{(t)} = - \sum_{1 \leq j \leq k} \tau_{v_t}(\delta_j) \cdot e_{\nabla_{\ell_{\text{ks}}}(\tau_{v_t}(a_j))} \end{cases},$$

with $e_{\nabla_{\ell_{\text{ks}}}}(w) := \langle \nabla_{\ell_{\text{ks}}} w, \mathbf{g}_{\text{ks}} \rangle - w$ for any $w \in \mathcal{R}_q$. After applying the $\otimes_{\psi_t^*}$ operation, the formula for obtaining $\mathcal{E}_t$ from $\mathcal{E}_{t-1}$ becomes (note the ψ_t^* stars)

$$\mathcal{E}_t = x^{s_t} \cdot \psi_t(\mathcal{E}_{t-1}) + (E_{\otimes}^{(t)} + x^{s_t} \cdot E_{\nabla, \psi_t^*}^{(t)}) + x^{s_t} \cdot \psi_t^*(E_{\text{ks}}^{(t)} + E_{\nabla_{\text{ks}, \tau_{v_t}}}^{(t)}) .$$

In the general case, let $\mathcal{A} = \{t \in \llbracket 1, n \rrbracket \mid \psi_t \notin \mathcal{S}\}$. Then, for any fixed $t \in \mathcal{A}$, [Algorithm 4.1](#) decomposes ψ_t as $\psi_t^* \circ \tau_{v_t} \circ \tau_{g^{w''}}^{q_t}$, where $\psi_t^* \in \mathcal{S}$, $\tau_{v_t} \neq \text{id}$ ($\Leftrightarrow v_t \neq 1$), and $\text{ak}^{\text{S-AUT}}$ contains a key corresponding to τ_{v_t} . Adapting the previous discussion to the case $q_t > 0$, it is easy to verify that for such $t \in \mathcal{A}$ we have

$$\mathcal{E}_t = x^{s_t} \cdot \psi_t(\mathcal{E}_{t-1}) + E_{\otimes}^{(t)} + x^{s_t} \cdot E_{\nabla, \psi_t^*}^{(t)} + x^{s_t} \cdot \psi_t^*(\mathcal{E}_{\text{ks}, v_t, q_t}^{(t)}) ,$$

where $\mathcal{E}_{\text{ks}, v_t, q_t}^{(t)} := (E_{\text{ks}}^{(t)} + E_{\nabla_{\text{ks}, \tau_{v_t}}}^{(t)}) + \sum_{a=1}^{q_t} \tau_{v_t} \tau_{g^{w''}}^{a-1} (E_{\text{ks}}^{(t,a)} + E_{\nabla_{\text{ks}, \tau_{g^{w''}}}}^{(t,a)})$ captures the errors from the required automorphism key switches when $t \in \mathcal{A}$. By abuse of notation, we let $\psi_t^* = \psi_t$ also when $t \notin \mathcal{A}$, i.e., when $\psi_t \in \mathcal{S}$. This allows us to prove by induction that the final error term of ACC is given by

$$\mathcal{E}_n = \sum_{t=1}^n \left(\prod_{a=t+1}^n x^{s_a u_{a+1} \cdots u_n} \right) \cdot \tau_{u_{t+1} \cdots u_n} \left(E_{\otimes}^{(t)} + x^{s_t} \cdot E_{\nabla, \psi_t^*}^{(t)} + \mathbb{1}_{t \in \mathcal{A}} \cdot x^{s_t} \cdot \psi_t^*(\mathcal{E}_{\text{ks}, v_t, q_t}^{(t)}) \right) .$$

It remains to bound the variance of $\mathcal{E}_n$ from this closed form.

First of all, the $E_{\otimes}^{(t)}$'s (resp. $E_{\text{ks}}^{(t,a)}$) can be considered as independent random samples of a random variable $E_{\otimes}$ (resp. E_{ks}), since the output distribution of the gadget decomposition is independent of the automorphism ψ_t^* , and the decomposed elements are combined with the bootstrapping keys errors. The same argument applies for the decomposition error terms $e_{\nabla_{\ell_u}}(w)$ for any $w \in \mathcal{R}_q$ and $u \in \{1, 2, \text{ks}\}$, however the images of the GLWE key $\psi(\delta_j)$ are not independent when t varies. In order to deal with this, we rewrite each $\psi(\delta_j) \cdot e_{\nabla_{\ell_u}}(w)$ as $\psi(\delta_j \cdot \psi^{-1}(e_{\nabla_{\ell_u}}(w)))$. The second key point is to notice that *all* subsequent automorphism applications (resp. multiplications by some power of x) are actually permutations (resp. rotations) of the error coefficients modulo $x^m - 1$, so that their variance is only multiplied once by C_∞ when reducing modulo Φ_m .

Thus, it is sufficient to bound the variance of $\langle \nabla_u w, \bar{\mathbf{e}} \rangle$ and $\delta_j \cdot \psi(e_{\nabla_{\ell_u}}(w))$, for any $u \in \{1, 2, \text{ks}\}$ and $w \in \mathcal{R}_q$. The result now follows from arguments similar to those in the proof of [Proposition 4.3](#). $\square$

Comparison with related works In [[LLW⁺24](#), Section 4, originally for NTRU] and concurrently in [[Lee24](#), Section 3], the authors describe an improved automorphism-based blind rotation, identified in [[LLW⁺24](#)] as “merging the symmetric

sets". Their method doubles the size of the keys, using both $\text{GLWE}_3^\oplus(x^{\pm s_i})$ for each $i \in \llbracket 1, n \rrbracket$. Algorithmically, this corresponds to the traversal method where the loop on $\epsilon \in \{\pm \epsilon_{\text{old}}\}$ is replaced by a choice of operand in the external product conditioned on ϵ . This removes all homomorphic applications of complex conjugation in the traversal method, which account for approximately 18% of the total number of the key switches when using an optimal window size. By contrast, the new $\mathcal{S}$ -parametrized method with $\mathcal{S} = \{\text{id}, \tau_{-1}\}$ also allows removing all complex conjugations, thus using *less* keys to achieve the same performance level. For instance, when $k = 1$, it needs $(2k + 1) \cdot n = 3n$ Gadget-GLWE ciphertexts, whereas following the approach of [LLW⁺24, Lee24] requires $2(k + 1) \cdot n = 4n$ Gadget-GLWE ciphertexts. More notably, setting $\mathcal{S} = \{\text{id}, \tau_{\pm g}\}$ allows removing about 46% of the key switches from [LMK⁺23] for about the same key size as what is needed in [LLW⁺24, Lee24] to remove only 18% of those. This highlights that our automorphism-parametrized external product provides a more effective solution than the approach in [LLW⁺24, Lee24].

Furthermore, aiming for a method similar to the $\mathcal{S}$ -parametrized approach with $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}\}$, but *via* a natural generalization of [LLW⁺24, Lee24],⁷ would require not only the keys $\text{GLWE}^\oplus(x^{\pm s_i})$, but also at least additional keys $\text{GLWE}^\oplus(x^{\pm g \cdot s_i})$. For $k = 1$, this already amounts to $8n$ Gadget-GLWE ciphertexts vs. $5n$ for the $\mathcal{S}$ -parametrized method. However, even with this allowed, it still does not reach the performance of the $\mathcal{S}$ -parametrized method: since no automorphism is applied during the external products, this generalized method cannot handle more than two *consecutive* gaps of type $\pm g$ without key switch. In this setting, our measurements show that this method brings only 60% of the performance gains provided by the $\mathcal{S}$ -parametrized method (see also Appendix C).

5 Analysis and Experiments

The complexity and noise analysis of the algorithms of this paper primarily reduce to evaluating the number of automorphism key switches performed, in addition to the n external products (whether automorphism-parametrized or not).

In previous works, this has been achieved using a rather loose worst-case upper bound (derived from) [LMK⁺23, Section 4.1], and with Monte Carlo simulations as in [WWL⁺24, Section 4.2] or [LLW⁺24, Figure 2]. In this work, we propose a theoretical framework for assessing the performance of our new automorphism-parametrized blind rotation, as well as prior automorphism-based algorithms. This framework is thoroughly validated through numerical experiments.

5.1 On Random Divisions of an Interval

We propose to reduce the problem of evaluating the number of automorphism key switches to analyzing the distribution of gaps in a *random cut* (with repetitions) of $\llbracket 0, B \rrbracket$,⁸ approximated by the continuous case $[0, B]$.

⁷ This has been recently formalized in a later work [ZWC25, Algorithm 6].

⁸ For instance, $B = N$ in Algorithm 3.1 and $B = N/2$ in Algorithms 3.2 and 4.1.

Consider n uniformly random variables $X_1, \dots, X_n$ sampled from $[0, B]$, and denote their ordered values by $X_{(i)}$, such that $0 \leq X_{(1)} \leq \dots \leq X_{(n)} \leq B$. For convenience, define $X_{(0)} = 0$ and $X_{(n+1)} = B$, capturing the starting and final points of the blind rotation loop. A *random cut* of $[0, B]$ is given by the $(n+1)$ random gaps $\Delta_i = X_{(i+1)} - X_{(i)}$, for $i \in \llbracket 0, n \rrbracket$, constrained by $\sum_{i=0}^n \Delta_i = B$.

Average maximum distance In this context (see e.g., [DN03, Section 6.4]), the joint probability density function of $\Delta_{i_1}, \dots, \Delta_{i_r}$, for any $r \in \llbracket 1, n \rrbracket$ and choice of the i_j 's, is known to be, for $\sum d_{i_j} \leq B$,

$$f(d_{i_1}, \dots, d_{i_r}) = \frac{n!}{B^r(n-r)!} \cdot \left(1 - \frac{d_{i_1} + d_{i_2} + \dots + d_{i_r}}{B}\right)^{n-r}.$$

This yields, by integrating r -times on $0 \leq \sum_{j=1}^r c_j \leq B$ [DN03, Equation 6.4.3],

$$\Pr[\Delta_{i_1} \geq c_1, \dots, \Delta_{i_r} \geq c_r] = \left(1 - \frac{\sum_{j=1}^r c_j}{B}\right)^n.$$

The probability of the maximum to be greater than $c \leq B$ is given by the *union* of all events $\Delta_i \geq c$. By the inclusion/exclusion principle, this writes

$$\Pr\left[\max_{0 \leq i \leq n} \{\Delta_i\} \geq c\right] = \sum_{\substack{1 \leq u \leq n+1 \\ \text{s.t. } uc \leq B}} (-1)^{u-1} \cdot \binom{n+1}{u} \left(1 - \frac{uc}{B}\right)^n. \quad (5.1)$$

The expectation is obtained by integrating this over all possible values of c , i.e.,

$$\begin{aligned} \mathbb{E}\left[\max_{0 \leq i \leq n} \{\Delta_i\}\right] &= \sum_{u=1}^{n+1} (-1)^{u-1} \cdot \binom{n+1}{u} \int_0^{B/u} \left(1 - \frac{ux}{B}\right)^n dx \\ &= \frac{B}{n+1} \cdot \sum_{u=1}^{n+1} \binom{n+1}{u} \frac{(-1)^{u-1}}{u} = \frac{B}{n+1} \cdot \sum_{u=1}^{n+1} \frac{1}{u}. \end{aligned} \quad (5.2)$$

The last equality may be proven by induction. Therefore, the average maximum gap can be approximated by $\frac{B}{n+1}(\ln(n+1) + \gamma)$, where $\gamma \approx 0.577$ is the Euler–Mascheroni constant, perfectly matching our experiments in the discrete case.

Average number of gaps of a given size Furthermore, we also need a more precise estimation of the number N_t of gaps of size $t \in \llbracket 0, B \rrbracket$. This can be obtained from the continuous case as follows.

Let (U_t) for $t \in \llbracket 0, B \rrbracket$ be a partition of $[0, B]$ enclosing integers with some offset $\omega = \omega_{n,B} \in]0, 1[$, i.e., $U_0 := [0, \omega]$, $U_t := [t-1+\omega, t+\omega]$ for $t \in \llbracket 1, B-1 \rrbracket$ and finally $U_B := [B-1+\omega, B]$. In addition, let $\tilde{N}_t = \#\{i \in \llbracket 0, n \rrbracket \mid \Delta_i \in U_t\}$; we heuristically assume that $\tilde{N}_t$ follows the same distribution as its discrete counterpart N_t . We rely on the following result, scaled from $[0, 1]$ to $[0, B]$.

Proposition 5.1 ([[Dar53](#), Equation 4.2]). *Let $W = \sum_{j=0}^n h(\Delta_j)$ for any integrable function h on $[0, B]$. Then*

$$\mathbb{E}[W] = (n+1) \cdot \int_0^B n \left(1 - \frac{r}{B}\right)^{n-1} h(r) \frac{dr}{B}.$$

Applying [Proposition 5.1](#) for $h_t(r) = \mathbb{1}_{U_t}(r)$, noting that $\tilde{N}_t = \sum_{j=0}^n \mathbb{1}_{U_t}(\Delta_j)$, we derive the following heuristic approximation of $\mathbb{E}[N_t]$, where $U_t = [u_t, v_t]$:

$$\mathbb{E}[N_t] \approx \mathbb{E}[\tilde{N}_t] = (n+1) \cdot \left[\left(1 - \frac{u_t}{B}\right)^n - \left(1 - \frac{v_t}{B}\right)^n \right]. \quad (5.3)$$

This approximation is close to $(n+1) \cdot (e^{-n \cdot u_t/B} - e^{-n \cdot v_t/B})$, which is generally easier to work with in practice due to its simpler exponential form.

It remains to determine the appropriate offset ω . To do so, we calibrate it so that the above-computed heuristic $\mathbb{E}[\tilde{N}_0] = (n+1) - (n+1)\left(1 - \frac{\omega}{B}\right)^n$ matches the formally proven $\mathbb{E}[N_0]$. In the discrete case, the average number of distinct values for sampling n integers amongst B values is given by $B - B(1 - \frac{1}{B})^n$. This is equivalent to having $\mathbb{E}[N_0] = n - B + B(1 - \frac{1}{B})^n$ collisions. Hence, we get

$$\omega = B - B \left(\frac{1 + B - B(1 - 1/B)^n}{n+1} \right)^{1/n}. \quad (5.4)$$

Remark 5.2. Using instead the approximations $\mathbb{E}[\tilde{N}_0] \approx (n+1) \cdot (1 - e^{-n\omega/B})$ and $\mathbb{E}[N_0] \approx n - B(1 - e^{-n/B})$, we can obtain a simpler expression for ω . Specifically, we get $(n+1) \cdot e^{-n/B \cdot \omega} \approx 1 + B(1 - e^{-n/B})$, so that $\omega \approx -\frac{1}{x} \ln\left(\frac{1-e^{-x}}{x}\right)$ for $x = \frac{n}{B}$. Assuming $x = o(1)$, its Taylor expansion writes

$$\omega \approx \frac{1}{2} - \frac{1}{24} \cdot \frac{n}{B} + \frac{1}{2880} \cdot \left(\frac{n}{B}\right)^3 - \dots,$$

which provides an indication of how close ω is to $\frac{1}{2}$.

Numerical validation We experimentally measured $\mathbb{E}[N_t]$ for two sets of parameters (n, B) : the ratio $\frac{n}{B}$ in [Figure 5.1\(a\)](#) is relevant for analyzing [Algorithm 3.1](#), whereas the ratio $\frac{n}{B}$ in [Figure 5.1\(b\)](#) pertains to the analysis of [Algorithms 3.2](#) and [4.1](#). In both cases, the observed results closely match the theoretical predictions, exhibiting the same shape. In particular, the results clearly show the exponential decay in the number of large gaps, which explains why a small set of automorphism keys suffices for automorphism-based blind rotations.

In addition, we observe significantly more gaps of size 1 compared to collisions. In practice, it is therefore more effective to handle gaps of size 1, e.g., with the $\mathcal{S}$ -parametrized method using $\mathcal{S} = \{\text{id}, \tau_g\}$, rather than gaining (roughly half of) the gaps of size 0 using complex conjugation.

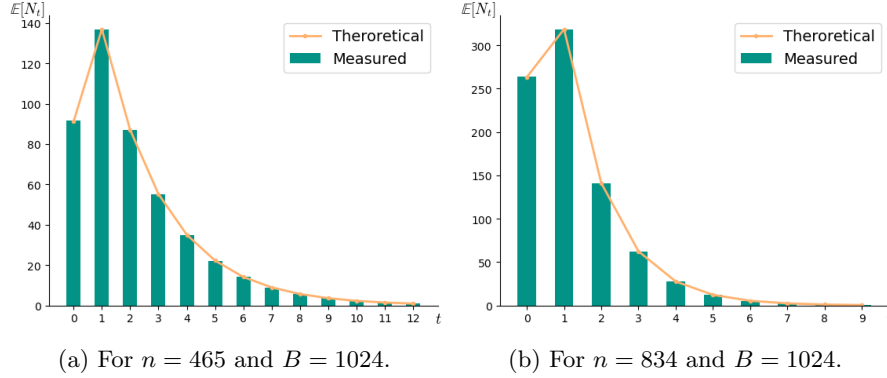


Fig. 5.1: Expectation for N_t , averaged over 10^4 samples of n (discrete logarithms) values modulo B vs. theoretical expectations obtained from Eqn. (5.3). Displayed values stop after the first t s.t. $\mathbb{E}[N_t] < 1$.

5.2 Theoretical Analysis of Automorphism-based Methods

We now apply the above discussion to analyze the average number of automorphism key switches required by Algorithms 3.1, 3.2 and 4.1. This readily provides average-case estimations of their computational complexity and noise growth.

Let $\kappa := \kappa(w)$ denote the random variable representing the number of key switches for a given window size w . Although κ also implicitly depends on n and $B = N$ or $\frac{N}{2}$, we omit these parameters since n and N are fixed across all methods. Using a maximal window size $w = B$ gives a lower bound on the average number of automorphism key switches required by the method, denoted by κ_∞ .

In this section, we provide explicit formulas closely approximating $\mathbb{E}[\kappa(w)]$ and $\mathbb{E}[\kappa_\infty]$ for all algorithms. We also discuss which choice of the window value w is optimal. In general, each algorithm is associated with a cost function $h(w, t)$, which represents the number of automorphism key switches required for a gap of size $t \in \llbracket 0, B \rrbracket$ using a window parameter w . Consequently, $\kappa(w)$ is simply computed as $\sum_{t=0}^B h(w, t) \cdot N_t$, where N_t denotes the number of gaps of size t .

Average number of key switches for Windowed-Horner methods The easiest case is that of the Windowed-Horner method, as the sets $I_t^\pm$ can be modeled directly⁹ as sampling n random values in $\llbracket 0, N \rrbracket$ with the cost function $h(w, t) = \lceil \frac{t}{w} \rceil$.

Proposition 5.3. *The expected number of automorphism key switches for a window of size w in Algorithm 3.1 using $\mathbf{ak}^{\text{HORN}}$ satisfies*

$$\mathbb{E}[\kappa^{\text{HORN}}(w)] \approx \left(1 + N(1 - e^{-n/N})\right) \cdot \frac{1}{1 - e^{-n/N \cdot w}}.$$

In particular, the best possible average value is $\mathbb{E}[\kappa_\infty^{\text{HORN}}] \approx 1 + N(1 - e^{-n/N})$.

⁹ Formally, using the bijection $g^a \mapsto a \in \llbracket 0, \frac{N}{2} - 1 \rrbracket$ and $-g^a \mapsto a + \frac{N}{2} \in \llbracket \frac{N}{2}, N - 1 \rrbracket$.

Proof. For a gap of size t , exactly $\lceil \frac{t}{w} \rceil$ automorphism key switches must be performed, thus $\mathbb{E}[\kappa^{\text{HORN}}(w)]$ is given by $\sum_{t=0}^N \lceil \frac{t}{w} \rceil \cdot \mathbb{E}[N_t]$, i.e., by Equation (5.3),

$$\mathbb{E}[\kappa^{\text{HORN}}(w)] \approx (n+1) \cdot \sum_{t=1}^N \left\lceil \frac{t}{w} \right\rceil \cdot \left(\left(1 - \frac{u_t}{N}\right)^n - \left(1 - \frac{v_t}{N}\right)^n \right).$$

Grouping the terms by values of $\lceil \frac{t}{w} \rceil$, and after canceling successive terms using that $u_t = v_{t-1}$ for $t \in \llbracket 1, N \rrbracket$ (the last term with v_N being 0), eventually yields

$$\begin{aligned} \mathbb{E}[\kappa^{\text{HORN}}(w)] &\approx (n+1) \cdot \sum_{k=0}^{\lceil N/w \rceil - 1} \left(1 - \frac{v_{kw}}{N}\right)^n \\ &\approx (n+1) e^{-n/N \cdot \omega} \cdot \frac{1 - e^{-n/N \cdot w \cdot \lceil N/w \rceil}}{1 - e^{-n/N \cdot w}}. \end{aligned}$$

The last expression is obtained by plugging $v_{kw} = kw + \omega$ and approximating each $\left(1 - \frac{v_{kw}}{N}\right)^n$ by $e^{-n/N \cdot v_{kw}} = e^{-n/N \cdot \omega} e^{-n/N \cdot wk}$. The numerator is bounded by $(1 - e^{-n})$ and $(1 - e^{-n(1+w/N)})$, both of which are astronomically close to 1. Finally, the offset ω is precisely defined so that $(n+1) \cdot e^{-\omega n/N}$ equals the expected number of distinct values plus one, i.e., $\mathbb{E}[\kappa_{\infty}^{\text{HORN}}] \approx 1 + N(1 - e^{-n/N})$. $\square$

For the Traversal Windowed-Horner method, the sets $I_t = I_t^+ \cup I_t^-$ can be modeled as sampling n random values in $\llbracket 0, \frac{N}{2} \rrbracket$, ignoring the signs. Since gap jumps always combine with possible sign changes, all gaps of size $t > 0$ in this model can be handled with exactly $h(w', t) = \lceil \frac{t}{w'} \rceil$ automorphism key switches, regardless of which I^ϵ is non-empty or processed first.

However, when both I_t^+ and I_t^- are non empty, we must account for additional sign changes. Luckily, the expected number of occurrences of this event is exactly given by the difference between the expected number of distinct values for n samples amongst N values vs. $N/2$, i.e., $N(1 - e^{-n/N}) - \frac{N}{2}(1 - e^{-2n/N})$. This leads to the following proposition.

Proposition 5.4. *The expected number of automorphism key switches for a window of size w' in Algorithm 3.2 using ak^{TRAV} satisfies*

$$\mathbb{E}[\kappa^{\text{TRAV}}(w')] \approx \frac{N}{2} \cdot (1 - e^{-n/N})^2 + \left(1 + \frac{N}{2}(1 - e^{-2n/N})\right) \cdot \frac{1}{1 - e^{-n/N \cdot 2w'}}.$$

In particular, the best possible average value is $\mathbb{E}[\kappa_{\infty}^{\text{TRAV}}] \approx 1 + N(1 - e^{-n/N})$.

As shown by Propositions 5.3 and 5.4, both methods converge to the same optimum. However, we can theoretically quantify the improvement brought by the Traversal method for a (fixed) equivalent amount of automorphism keys, i.e., for $w = 2w'$. In that case, the difference simplifies to

$$\mathbb{E}[\kappa^{\text{HORN}}(w)] - \mathbb{E}[\kappa^{\text{TRAV}}(w')] \approx \frac{N}{2} (1 - e^{-n/N})^2 \cdot \left(\frac{1}{1 - e^{-n/N \cdot w}} - 1 \right),$$

which is strictly positive and decreases towards 0, as expected. This allows the corresponding ratio to be expressed directly as

$$\begin{aligned} \frac{\mathbb{E}[\kappa^{\text{TRAV}}(w')]}{\mathbb{E}[\kappa^{\text{HORN}}(w)]} &\approx 1 - \frac{1}{2} \cdot \frac{N(1 - e^{-n/N})^2}{1 + N(1 - e^{-n/N})} \cdot e^{-n/N \cdot w} \\ &\approx 1 - \frac{1}{2} \cdot (1 - e^{-n/N}) \cdot e^{-n/N \cdot w}, \end{aligned}$$

which perfectly aligns with the experimental results presented in [Table 3.1](#).

Average number of key switches for the automorphism-parametrized method As for the automorphism-parametrized method, the number of automorphism key switches depends on the specific set $\mathcal{S}$ of automorphisms utilized.

We first consider the simplest case where $\mathcal{S}$ contains all automorphisms τ_{g^k} for k up to $K \geq 0$, and is *symmetric*, i.e., $\tau_a \in \mathcal{S} \Rightarrow \tau_{-a} \in \mathcal{S}$. This basically means that all gap jumps of size at most K , including those involving possible sign changes, can be handled using our new parametrized external product.

Proposition 5.5. *Suppose $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}, \dots, \tau_{\pm g^K}\}$ for $K \geq 0$. The expected number of automorphism key switches for a window of size w'' in [Algorithm 4.1](#) using $\text{ak}^{\text{S-AUT}}[0] \cup \text{ak}^{\text{S-AUT}}[1 \dots w'']$ satisfies*

$$\mathbb{E}[\kappa^{\text{S-AUT}}(w'')] \approx \frac{1}{2} + \left(1 + \frac{N}{2}(1 - e^{-2n/N})\right) \cdot e^{-K \cdot 2n/N} \cdot \frac{1}{1 - e^{-n/N \cdot 2w''}}.$$

This tends towards $\mathbb{E}[\kappa_{\infty}^{\text{S-AUT}}] \approx \frac{1}{2} + \left(1 + \frac{N}{2}(1 - e^{-2n/N})\right) \cdot e^{-K \cdot 2n/N}$ with w'' .

In other words, [Proposition 5.5](#) shows that increasing K by 1 essentially reduces the number of automorphism key switches by a factor of $e^{2n/N}$, which is approximately 2.26 for $n = 834$ and $N = 2048$ (resp. 2.48 for $n = 465$ and $N = 1024$). This matches the measurements presented in [Table 5.4](#).

Remark 5.6. The case $K = 0$, corresponding to $\mathcal{S} = \{\text{id}, \tau_{-1}\}$, is *computationally* equivalent to (though requiring *less* keys than) the proposal in [\[LLW⁺24\]](#). Indeed, both settings eliminate precisely the homomorphic complex conjugations.

Proof (of [Proposition 5.5](#)). The modeling is the same as for the Traversal method with $B = \frac{N}{2}$. Due to the shape of $\mathcal{S}$, all gaps of size $t \leq K$, including a possible sign change, incur no cost. For gaps of size $t > K$, $\lceil \frac{t-K}{w''} \rceil$ automorphism key switches are required (followed e.g., by a new external product parametrized by $\tau_{\pm g^K}$). Thus, the expected number of automorphism key switches is

$$\mathbb{E}[\kappa^{\text{S-AUT}}(w'')] \approx (n+1) \cdot \sum_{t=K+1}^{N/2} \left\lceil \frac{t-K}{w''} \right\rceil \cdot \left(\left(1 - \frac{u_t}{N/2}\right)^n - \left(1 - \frac{v_t}{N/2}\right)^n \right).$$

Using the same reasoning as before, this simplifies to the right-hand side of the result. Finally, since only the “positive” part of $\text{ak}^{\text{S-AUT}}$ is used, applying a sign change using $\text{ak}^{\text{S-AUT}}[0]$ ([Algorithm 4.1, Line 15](#)) is required half of the time. $\square$

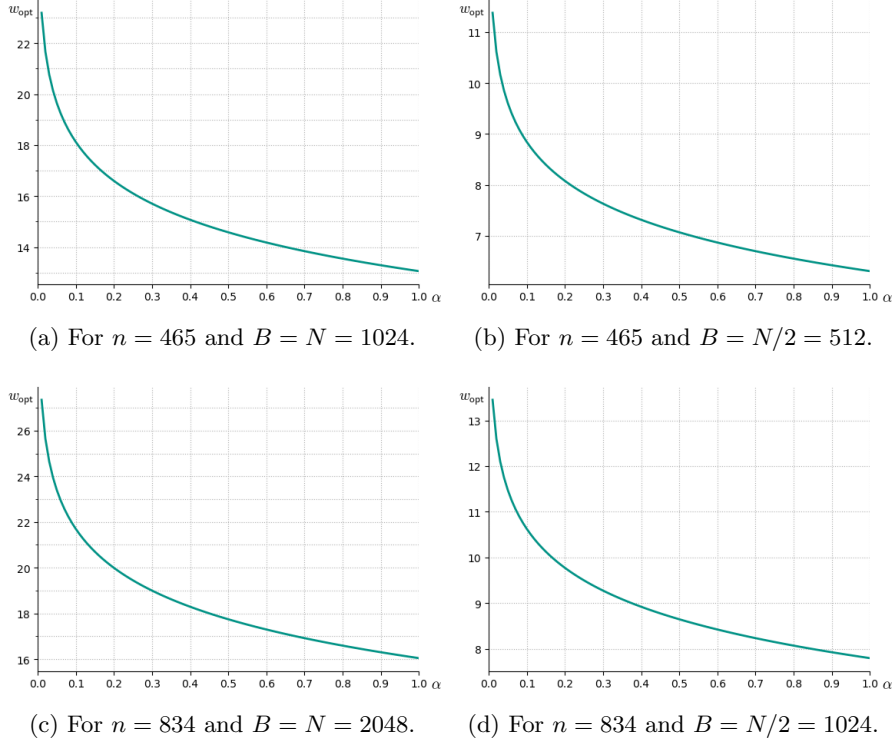


Fig. 5.2: Optimal window size w_{opt} , depending on the average distance α to κ_{∞} . Using $B = N$ corresponds to Algorithm 3.1 and [LMK⁺23], whereas $B = N/2$ captures Algorithm 3.2 (w'_{opt}), Algorithm 4.1 for $\mathcal{S} = \{\tau_{\pm 1}\}$ and [LLW⁺24].

We also explore various tradeoffs where $\mathcal{S}$ is not symmetrical. In particular, if $\mathcal{S}$ only lacks τ_{-1} to be symmetrical, the situation is similar to Proposition 5.4 for the Traversal method: automorphism key switches using $\text{ak}^{\text{S-AUT}}[0]$ are only required when both $I_t^{\pm}$ are non-empty, so $\frac{N}{2} \cdot (1 - e^{-n/N})^2$ is simply added.

The situation is trickier in general. We discuss the case $\mathcal{S} = \{\text{id}, \tau_g\}$, as other cases appear to be of limited interest. Notably, this is the first scenario where both the “negative” and “positive” parts of $\text{ak}^{\text{S-AUT}}$ are required. As before, we must add the complex conjugation count, i.e., $\frac{N}{2} \cdot (1 - e^{-n/N})^2$. However, we also must account for size-1 gaps that necessitate a sign change. Empirically, we observed that $\theta \approx 60\%$ of these size-1 gaps incur an additional sign change cost.

On the optimal window size In previous works, the optimal window size is guesstimated as the point where some experimental graph sufficiently flattens. For instance, [LMK⁺23, Figure 3] suggests using $w = 10$, while in an equivalent setting [LLW⁺24, Figure 2(b)] instead suggests using $w = 20$.

We propose a more robust definition of this optimal window size. Intuitively, the optimal window size should be set such that no greater gaps occur on average. More formally, we define w_{opt} as the smallest w s.t. $\mathbb{E}[\kappa(w)] \leq \mathbb{E}[\kappa_\infty] + \alpha$, where $0 < \alpha \leq 1$ is fixed. Thus, let w_{opt} (resp. w'_{opt} , w''_{opt}) denote the corresponding optimal window value w.r.t. [Algorithm 3.1](#) (resp. [Algorithm 3.2](#), resp. [Algorithm 4.1](#) for a symmetric set of automorphisms $\mathcal{S} = \{\tau_{\pm g^k} \mid 0 \leq k \leq K\}$). Solving directly the condition for w_{opt} , w'_{opt} using [Propositions 5.3](#) and [5.4](#) yields the equivalent expression

$$w_{\text{opt}}^{(\prime)} = \left\lceil \frac{B}{n} \cdot \ln \left(1 + \frac{1 + B(1 - e^{-n/B})}{\alpha} \right) \right\rceil, \quad (5.5)$$

where $B = N$ for w_{opt} and $B = N/2$ for w'_{opt} . Likewise, using [Proposition 5.5](#), we obtain that for a given $K \geq 0$, w''_{opt} verifies

$$w''_{\text{opt}} = \left\lceil \frac{N}{2n} \ln \left(1 + \frac{1 + N/2(1 - e^{-2n/N})}{\alpha} \cdot e^{-K \cdot 2n/N} \right) \right\rceil \approx w'_{\text{opt}} - K. \quad (5.6)$$

Curves for $w_{\text{opt}}^{(\prime)}$ (omitting the ceiling) are given in [Figure 5.2](#). We remark that $w''_{\text{opt}} = w'_{\text{opt}}$ when $K = 0$, so that the figures with $B = N/2$ (w'_{opt}) also encompass optimal window sizes for [Algorithm 4.1](#) (w''_{opt}) with $\mathcal{S} = \{\tau_{\pm 1}\}$ as well as [\[LLW⁺24\]](#) (see [Remark 5.6](#)). For example, for $n = 465$ and $N = 1024$, [Figure 5.2\(a\)](#) shows that choosing $w_{\text{opt}} = 20$ ensures $\alpha \geq 0.043$, whereas [Figure 5.2\(b\)](#) indicates that setting $w'_{\text{opt}} = 8$ as in [\[LLW⁺24\]](#) only yields $\alpha \geq 0.215$. Conversely, targeting $\alpha = 0.25$ yields resp. $w_{\text{opt}} = \lceil 16.1 \rceil$ and $w'_{\text{opt}} = \lceil 7.8 \rceil$, according to [Equation \(5.5\)](#). This also supports the intuition that $w_{\text{opt}} \approx 2w'_{\text{opt}}$, since the average maximum gap is roughly halved from $\frac{N/2}{n/2} \ln(n/2)$ to $\frac{N/2}{n} \ln n$.

Remark 5.7. Using the approximation $1 + B(1 - e^{-n/B}) \approx (n+1) \cdot e^{-n\omega_{n,B}/B}$ (see [Remark 5.2](#)), and assuming $\omega_{n,B} < \frac{1}{2}$ and $n < B$, we easily obtain much simpler—though not fully accurate—expressions:

$$w_{\text{opt}} \approx \left\lceil \frac{N}{n} \cdot \ln \frac{n+1}{\alpha} \right\rceil, \quad w'_{\text{opt}} \approx \left\lceil \frac{N}{2n} \cdot \ln \frac{n+1}{\alpha} \right\rceil, \\ w''_{\text{opt}} \approx \left\lceil \frac{N}{2n} \cdot \ln \frac{n+1}{\alpha} \right\rceil - K.$$

As expected, these values are close to the average maximum gap when $\alpha = 1$. This yields a direct explanation of why small window sizes suffice in practice and clearly shows that the optimal window grows logarithmically as α approaches 0.

Theoretical summary The performance of automorphism-based blind rotation algorithms is summarized in [Table 5.3](#) according to [Propositions 5.3](#), [5.4](#) and [5.5](#). As throughout this paper, the results are given for null or invertible mask components. We also consider in the `ak` column that $w' = w/2 = w'' + K$ (see [Remark 5.7](#)) in order to ease comparisons. The number of (automorphism-parametrized) external products is always exactly n and is therefore omitted.

Table 5.3: Comparison of automorphism-based blind rotations w.r.t. evaluation key material and best average number of automorphism key switches. For concision, $A_{n/N}$ denotes $e^{-n/N}$, and experimental observations suggest $\theta \approx 60\%$.

	Keys size ($\# \text{GLWE}^\nabla$)		Avg. number of aut. key switches ($\mathbb{E}[\kappa_\infty]$, i.e., using $w_{\text{opt}}^{(\iota, \iota')}$)
	bsk	ak	
Telescoping (Fig. 2.1(b))	$(k+1)n$	kN	$\min\{n, N\}$
Wind.-Horner (Alg. 3.1)	$(k+1)n$	$k(w+1)$	$N \cdot (1 - A_{n/N})$
Traversal (Alg. 3.2)	$(k+1)n$	$k(2w'+1)$	$N \cdot (1 - A_{n/N})$
[LLW ⁺ 24, Alg. 2],	$2(k+1)n$	kw'	$\frac{N}{2} \cdot (1 - A_{n/N}^2)$
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}\}$	$(2k+1)n$	$k(w'+1)$	$\frac{N}{2} \cdot (1 - A_{n/N}^2)$
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_g\}$	$(2k+1)n$	$k(2w'-1)$	$N \cdot (1 - A_{n/N}) - \frac{N}{2} \cdot \theta (1 - A_{n/N}^2)^2$
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_{\pm g}\}$	$(3k+1)n$	kw'	$\frac{N}{2} \cdot (A_{n/N}^2 - A_{n/N}^4 + (1 - A_{n/N})^2)$
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}\}$	$(4k+1)n$	kw'	$\frac{N}{2} \cdot (1 - A_{n/N}^2) \cdot A_{n/N}^2$
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}, \tau_{\pm g^2}\}$	$(6k+1)n$	$k(w'-1)$	$\frac{N}{2} \cdot (1 - A_{n/N}^2) \cdot A_{n/N}^4$
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid \delta < K\}$	$(2Kk+1)n$	$k(w'-K+2)$	$\frac{N}{2} \cdot (1 - A_{n/N}^2) \cdot A_{n/N}^{2(K-1)}$

5.3 Numerical Measurements

To demonstrate the impact of our techniques on automorphism-based blind rotations, we conducted experiments to measure the average number of (extended and classical) external products and automorphism key switches required by each of our new variants, comparing their performance against [LMK⁺23, LLW⁺24].

The results are summarized in Table 5.4 for the two different parameter sets, corresponding to those in Section 3, however using $n = 465$ instead of $n = 458$ for the first set, as in [LLW⁺24]. The optimal window values are computed from Equations (5.5) and (5.6) under the rather stringent¹⁰ requirement that the distance between $\kappa(w_{\text{opt}}^{(\iota, \iota')})$ and the corresponding κ_∞ does not exceed $\alpha = 0.25$ on average. The number of automorphism key switches and operation counts are then averaged over 10^4 random mask components in $(\mathbb{Z}/2N\mathbb{Z})^\times$. Although $\otimes$ and $\otimes_{\mathcal{S}}$ are counted separately to highlight the use of the new parametrized external product, both operations are strictly equivalent computationally.

The results indicate that, with the same increase of the bootstrapping keys as in [LLW⁺24], Algorithm 4.1 with $\mathcal{S} = \{\text{id}, \tau_{\pm g}\}$ already requires 37.1% (resp. 35.5%) fewer key switches compared to [LLW⁺24] and 49.1% (resp. 46.4%) fewer compared to [LMK⁺23]). More strikingly, with keys that are 25% smaller than in [LLW⁺24], Algorithm 4.1 using the asymmetric set $\mathcal{S} = \{\text{id}, \tau_g\}$ already outperforms [LLW⁺24] by over 14.0% (resp. 13.3%). In particular, this clearly demonstrates an advantage over using $\mathcal{S} = \{\tau_{\pm 1}\}$, as predicted from Figure 5.1.

Further trade-offs are also possible. With only a moderate increase of the size of $\text{bsk}^{\mathcal{S}\text{-AUT}}$, i.e., multiplied by 2.5 compared to [LMK⁺23] instead of 2 as in [LLW⁺24], Algorithm 4.1 with $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}\}$ achieves a reduction of the

¹⁰ E.g., the choice $w'' = 8$ made in [LLW⁺24] could correspond to any $\alpha \in [0.22, 0.53]$.

Table 5.4: Experimentally measured number of regular ($\otimes$), automorphism-parametrized ($\otimes_S$) external products, and automorphism key switches (KS) for automorphism-based blind rotations, averaged over 10^4 samples. The optimal window values $w_{\text{opt}}^{(t, \prime\prime)}$ are computed for $\alpha = 0.25$.

(a) Parameters $n = 465$ and $N = 1024$, $k = 1$.

Method	$w_{\text{opt}}^{(t, \prime\prime)}$	Keys ($\# \text{GLWE}^\nabla$)	$\#(\otimes)$	$\#(\otimes_S)$	$\#(\text{KS})$
Windowed-Horner (Alg. 3.1, [LMK ⁺ 23])	17	$2n + 18$	465	0	375.8
Traversal Windowed-Horner (Alg. 3.2)	± 8	$2n + 17$	465	0	375.0
[LLW ⁺ 24, Alg. 2]	8	$4n + 8$	465	0	306.1
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}\}$	8	$3n + 9$	210	255	306.6
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_g\}$	± 7	$3n + 15$	306	159	263.1
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_{\pm g}\}$	7	$4n + 8$	159	306	192.5
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_g\}$	± 7	$4n + 15$	91	374	195.4
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}\}$	7	$5n + 8$	91	374	124.3
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_{\pm g}, \tau_{\pm g^2}\}$	6	$6n + 7$	159	306	118.8
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}, \tau_{\pm g^2}\}$	6	$7n + 7$	91	374	50.6
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 3\}$	5	$9n + 6$	91	374	20.9
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 4\}$	4	$11n + 5$	91	374	9.0
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 5\}$	3	$13n + 4$	91	374	4.3
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 6\}$	3	$15n + 4$	91	374	1.9
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 7\}$	2	$17n + 3$	91	374	1.5

(b) Parameters $n = 834$ and $N = 2048$, $k = 1$.

Method	$w_{\text{opt}}^{(t, \prime\prime)}$	Keys ($\# \text{GLWE}^\nabla$)	$\#(\otimes)$	$\#(\otimes_S)$	$\#(\text{KS})$
Windowed-Horner (Alg. 3.1, [LMK ⁺ 23])	20	$2n + 21$	834	0	688.5
Traversal Windowed-Horner (Alg. 3.2)	± 10	$2n + 21$	834	0	686.4
[LLW ⁺ 24, Alg. 2]	10	$4n + 10$	834	0	571.4
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}\}$	10	$3n + 11$	376	458	571.9
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_g\}$	± 9	$3n + 19$	264	570	495.5
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_{\pm g}\}$	9	$4n + 10$	264	570	368.7
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_g\}$	± 9	$4n + 10$	149	685	380.8
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}\}$	9	$5n + 10$	149	685	254.0
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\text{id}, \tau_{\pm g}, \tau_{\pm g^2}\}$	8	$6n + 9$	263	571	227.2
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}, \tau_{\pm g^2}\}$	8	$7n + 9$	149	685	112.5
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 3\}$	7	$9n + 8$	149	685	50.1
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 4\}$	6	$11n + 7$	149	685	23.0
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 5\}$	5	$13n + 6$	149	685	10.9
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 6\}$	4	$15n + 5$	149	685	5.4
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 7\}$	3	$17n + 4$	149	685	3.1
$\mathcal{S}$ -Aut, $\mathcal{S} = \{\tau_{\pm g^\delta} \mid 0 \leq \delta \leq 8\}$	2	$19n + 3$	149	685	1.9

number of key switches of more than 59.4% (resp. 55.5%) relative to [LLW⁺24] and 66.9% (resp. 63.1%) relative to [LMK⁺23]). Moreover, the last rows of the tables illustrate the capability of our new $\mathcal{S}$ -parametrized method to approach the computational efficiency of the AP bootstrapping while maintaining reasonably sized keys. Indeed, considering keys only up to 9 times larger w.r.t. [LMK⁺23], the average number of key switches can be squeezed down to only 2 or 3, whereas AP keys are more than 2 orders of magnitude larger for comparable performance. As the convergence is quite fast, the set $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}, \tau_{\pm g^2}\}$ yields an appealing middle ground: with a $3.5\times$ increase in $\text{bsk}^{\mathcal{S}\text{-AUT}}$ compared to [LMK⁺23], it eliminates 86.5% (resp. 83.6%) of the key switches of the Traversal Windowed-Horner method.

As a final remark, we emphasize that while the proposed variants imply increasing the size of the bootstrapping keys compared to [GINX16] or [LMK⁺23], only $(k+1)$ GLWE ^{∇} ciphertexts are ever needed for computing each external product, whether parametrized or not. Those can easily be prefetched as soon as the (mod-switched) mask components are known and sorted, therefore the bandwidth requirements for our methods remain unchanged.

About absolute time improvements for the bootstrapping In each of the methods presented above, the entire bootstrapping procedure consists of n external products alongside the automorphism key switches counted by our theoretical complexity analysis in Propositions 5.3, 5.4 and 5.5. We obviously did not report everywhere this constant number of external products (whether automorphism-parametrized or not). However, from the discussion in Section 2 regarding the respective cost ratio of a key switch and an external product, it is relatively straightforward to derive the absolute runtime improvement by our method.

For example, according to Table 5.4(a), the Windowed-Horner method (Algorithm 3.1, [LMK⁺23, Algorithm 7]) incurs a total bootstrapping cost of (at least) $\frac{376 \cdot 5/8 + 465}{465} \approx 1.51$ times the cost of computing $n = 465$ external products as would be the case using binary GINX; i.e., a 51% overhead. In contrast, our method using $\mathcal{S} = \{\tau_{\pm 1}, \tau_{\pm g}, \dots, \tau_{\pm g^6}\}$ reduces this to $\frac{2 \cdot 5/8 + 465}{465} \approx 1.003$, effectively eliminating this overhead —hence the title, and achieving (at least) a 34% *overall improvement* in both performance and noise growth over [LMK⁺23], and up to 38% for $k = 1$ and $\ell = 1$, as mostly used in TFHE-rs. Identical results are obtained for the second parameter set in Table 5.4(b).

References

- AP14. Jacob Alperin-Sheriff and Chris Peikert. Faster bootstrapping with polynomial error. In J. A. Garay and R. Gennaro, editors, *Advances in Cryptology — CRYPTO 2014, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 297–314. Springer, 2014. doi:10.1007/978-3-662-44371-2_17.
- BCG⁺24. Mariya Georgieva Belorgey, Sergiu Carпов, Nicolas Gama, Sandra Guasch, and Dimitar Jetchev. Revisiting key decomposition techniques for FHE: Simpler, faster and more generic. In K.-M. Chung and Y. Sasaki, editors, *Advances in Cryptology — ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 176–207. Springer, Singapore, 2024. doi:10.1007/978-981-96-0875-1_6.
- BDF18. Guillaume Bonnoron, Léo Ducas, and Max Fillinger. Large FHE gates from tensored homomorphic accumulator. In A. Joux, A. Nitaj, and T. Rachidi, editors, *Progress in Cryptology — AFRICACRYPT 2018*, volume 10831 of *Lecture Notes in Computer Science*, pages 217–251. Springer, Cham, 2018. doi:10.1007/978-3-319-89339-6_13.
- BIP⁺22. Charlotte Bonte, Iliia Iliashenko, Jeongeun Park, Hilder V. L. Pereira, and Nigel P. Smart. FINAL: Faster FHE instantiated with NTRU and LWE. In S. Agrawal and D. Lin, editors, *Advances in Cryptology — ASIACRYPT 2022, Part II*, volume 13792 of *Lecture Notes in Computer Science*, pages 188–215. Springer, Cham, 2022. doi:10.1007/978-3-031-22966-4_7.
- BJ24. Olivier Bernard and Marc Joye. Approximate CRT-based gadget decomposition and application to TFHE blind rotation. Cryptology ePrint Archive, 2024. URL: <https://ia.cr/2024/909>.
- BJSW25. Olivier Bernard, Marc Joye, Nigel Smart, and Michael Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In S. Fehr and P.-A. Fouque, editors, *Advances in Cryptology — EUROCRYPT 2025, Part VIII*, volume 15608 of *Lecture Notes in Computer Science*, pages 181–211. Springer, Cham, 2025. doi:10.1007/978-3-031-91101-9_7.
- CGGI20. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020. doi:10.1007/s00145-019-09319-x.
- CJP21. Ilaria Chillotti, Marc Joye, and Pascal Paillier. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In S. Dolev et al., editors, *Cyber Security Cryptography and Machine Learning (CSCML 2021)*, volume 12716 of *Lecture Notes in Computer Science*, pages 1–19. Springer, 2021. doi:10.1007/978-3-030-78086-9_1.
- Dar53. Donald A. Darling. On a class of problems related to the random division of an interval. *The Annals of Mathematical Statistics*, 24(2):239–253, 1953. doi:10.1214/aoms/1177729030.
- DM15. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In E. Oswald and M. Fischlin, editors, *Advances in Cryptology — EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640. Springer, Berlin, Heidelberg, 2015. doi:10.1007/978-3-662-46800-5_24.

- DN03. Herbert A. David and Haikady N. Nagaraja. *Order Statistics*. Wiley Series in Probability and Statistics. John Wiley & Sons, Inc., 3rd edition, 2003. [doi:10.1002/0471722162](https://doi.org/10.1002/0471722162).
- Gen10. Craig Gentry. Computing arbitrary functions of encrypted data. *Communications of the ACM*, 53(3):97–105, 2010. [doi:10.1145/1666420.1666444](https://doi.org/10.1145/1666420.1666444).
- GINX16. Nicolas Gama, Malika Izabachène, Phong Q. Nguyen, and Xiang Xie. Structural lattice reduction: Generalized worst-case to average-case reductions and homomorphic cryptosystems. In M. Fischlin and J.-S. Coron, editors, *Advances in Cryptology — EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 528–558. Springer, Berlin, Heidelberg, 2016. [doi:10.1007/978-3-662-49896-5_19](https://doi.org/10.1007/978-3-662-49896-5_19).
- GSW13. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In R. Canetti and J. A. Garay, editors, *Advances in Cryptology — CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92. Springer, Berlin, Heidelberg, 2013. [doi:10.1007/978-3-642-40041-4_5](https://doi.org/10.1007/978-3-642-40041-4_5).
- HKLS24. Deokhwa Hong, Young-Sik Kim, Yongwoo Lee, and Eunyoung Seo. A new fine tuning method for FHEW/TFHE bootstrapping with IND-CPA^D security. *Cryptology ePrint Archive*, 2024. URL: <https://ia.cr/2024/1052>.
- Joy22. Marc Joye. SoK: Fully homomorphic encryption over the [discretized] torus. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):661–692, 2022. [doi:10.46586/tches.v2022.i4.661-692](https://doi.org/10.46586/tches.v2022.i4.661-692).
- JP22. Marc Joye and Pascal Paillier. Blind rotation in fully homomorphic encryption with extended keys. In S. Dolev, J. Katz, and A. Meisels, editors, *Cyber Security, Cryptology, and Machine Learning: 6th International Symposium (CSCML 2022)*, volume 13301 of *Lecture Notes in Computer Science*, pages 1–18. Springer, Cham, 2022. [doi:10.1007/978-3-031-07689-3_1](https://doi.org/10.1007/978-3-031-07689-3_1).
- JW22. Marc Joye and Michael Walter. Liberating TFHE: Programmable bootstrapping with general quotient polynomials. In M. Brenner, A. Costache, and K. Rohloff, editors, *WAHC’22: Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 1–11. Association for Computing Machinery, New York, 2022. [doi:10.1145/3560827.3563376](https://doi.org/10.1145/3560827.3563376).
- Lee24. Yongwoo Lee. LMKCDEY revisited: Speeding up blind rotation with signed evaluation keys. *Mathematics*, 12(18), 2024. [doi:10.3390/math12182909](https://doi.org/10.3390/math12182909).
- LLW⁺24. Zhihao Li, Xianhui Lu, Zhiwei Wang, Ruida Wang, Ying Liu, Yinhang Zheng, Lutan Zhao, Kunpeng Wang, and Rui Hou. Faster NTRU-based bootstrapping in less than 4 ms. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):418–451, 2024. [doi:10.46586/tches.v2024.i3.418-451](https://doi.org/10.46586/tches.v2024.i3.418-451).
- LMK⁺23. Yongwoo Lee, Daniele Micciancio, Andrey Kim, Rakyong Choi, Maxim Deryabin, Jieun Eom, and Donghoon Yoo. Efficient FHEW bootstrapping with small evaluation keys, and application to threshold homomorphic encryption. In C. Hazay and M. Stam, editors, *Advances in Cryptology — EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 227–256. Springer, Cham, 2023. [doi:10.1007/978-3-031-30620-4_8](https://doi.org/10.1007/978-3-031-30620-4_8).

- LPR10. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In H. Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010. doi:10.1007/978-3-642-13190-5_1.
- LS15. Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75:565–599, 2015. doi:10.1007/s10623-014-9938-4.
- MKMS24. Gabrielle De Micheli, Duhyeon Kim, Daniele Micciancio, and Adam Suhl. Faster amortized FHEW bootstrapping using ring automorphisms. In Q. Tang and V. Teague, editors, *Public-Key Cryptography — PKC 2024, Part IV*, volume 14604 of *Lecture Notes in Computer Science*, pages 322–353. Springer, Cham, 2024. doi:10.1007/978-3-031-57728-4_11.
- MP21. Daniele Micciancio and Yuriy Polyakov. Bootstrapping in FHEW-like cryptosystems. In M. Brenner, R. Player, and K. Rohloff, editors, *9th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC 2021)*, pages 17–28. ACM Press, 2021. doi:10.1145/3474366.3486924.
- RAD78. Ronald L. Rivest, Len Adleman, and Michael L. Dertouzos. On data banks and privacy homomorphisms. In R. A. DeMillo et al., editors, *Foundations of Secure Computation*, pages 169–179. Academic Press, 1978. Available at <https://people.csail.mit.edu/rivest/pubs.html#RAD78>.
- Reg09. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):34:1–34:40, 2009. doi:10.1145/1568318.1568324.
- SSTX09. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient public key encryption based on ideal lattices. In M. Matsui, editor, *Advances in Cryptology — ASIACRYPT 2009*, volume 5912 of *Lecture Notes in Computer Science*, pages 617–635. Springer, Berlin, Heidelberg, 2009. doi:10.1007/978-3-642-10366-7_36.
- WWL⁺24. Ruida Wang, Yundi Wen, Zhihao Li, Xianhui Lu, Benqiang Wei, Kun Liu, and Kunpeng Wang. Circuit bootstrapping: Faster and smaller. In M. Joye and G. Leander, editors, *Advances in Cryptology — EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 342–372. Springer, Cham, 2024. doi:10.1007/978-3-031-58723-8_12.
- XZD⁺23. Binwu Xiang, Jiang Zhang, Yi Deng, Yiran Dai, and Dengguo Feng. Fast blind rotation for bootstrapping FHEs. In H. Handschuh and A. Lysyanskaya, editors, *Advances in Cryptology — CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 3–36. Springer, Cham, 2023. doi:10.1007/978-3-031-38551-3_1.
- Zam22. Zama. TFHE-rs: A pure Rust implementation of the TFHE scheme for boolean and integer arithmetics over encrypted data, 2022. URL: <https://github.com/zama-ai/tfhe-rs>.
- ZWC25. Qi Zhang, Mingqiang Wang, and Xiaopeng Cheng. Faster FHEW bootstrapping with adaptive key update. *Cryptology ePrint Archive*, 2025. URL: <https://ia.cr/2025/381>.

A Dealing With Arbitrary Mask Components

In the general case, the modulus switching, as part of the (programmable) bootstrapping, does not directly yield mask components $\tilde{a}_i \in (\mathbb{Z}/m\mathbb{Z})^\times$ correspond-

ing to automorphisms, or $\tilde{a}_i = 0$. Several techniques have been proposed to circumvent this:

1. changing the base ring to a circulant ring [BDF18] or a prime conductor cyclotomic ring of integers [MKMS24];
2. rewriting $\sum_i \tilde{a}_i s_i$ as $\sum_i \tilde{a}_i^* s_i^*$, where $\tilde{a}_i^* \in (\mathbb{Z}/m\mathbb{Z})^\times \cup \{0\}$, plus a correcting step implementing a trade-off between noise growth/performance and key material [LMK⁺23, Sections 3.2.1 and 3.2.2];
3. adapting the modulus switch to round to either 0 or directly to invertible elements [LMK⁺23, Section 3.3], or further, directly to elements of a cyclic subgroup thereof [WWL⁺24].

The first approach eludes the problem altogether. We therefore focus below on the two latter approaches, viewed as *pre-conditioning* algorithms for the LWE input ciphertext and keys before entering the blind rotation. They both suppose $m = 2N$ is a power of two so that every odd integer has an inverse modulo m .

A.1 Rewriting the inner product

Two reorganizations of the inner product are given in [LMK⁺23, Sections 3.2.1 and 3.2.2].

The first method [LMK⁺23, Algorithm 4] simply consists in setting the new mask components as $\tilde{a}_i^* = \tilde{a}_i - 1$ if $\tilde{a}_i \neq 0$ is even, and $\tilde{a}_i$ otherwise; then, additional external products, one per even value $\tilde{a}_i$, need to be applied to the accumulator.¹¹ This incurs on average a significant computational overhead of about $n/2$ external products, which can be strictly limited to $n/2 + 1$ using an extra bootstrapping key $\text{bsk}_{\text{nsum}}^{\text{AUT}}$ encrypting $x^{-\sum_i s_i}$ and allowing to flip the parity of all mask components at once.

The second method [LMK⁺23, Algorithm 5] approximately doubles the size of the bootstrapping keys by including

$$\text{bsk}^{\text{AUT}*} := \left\{ \text{GLWE}_{\mathfrak{d}}^{\otimes}(x^{s_i + s_{i+1}}) \mid i \in \llbracket 1, n-1 \rrbracket \right\}.$$

Assuming without loss of generality that $\tilde{a}_1$ is odd, then iteratively when $\tilde{a}_{i+1}$ is even, $\tilde{a}_{i+1}^*$ is set to $(\tilde{a}_{i+1} - \tilde{a}_i^*)$ and the bootstrapping key for position i is modified to $\text{bsk}^{\text{AUT}*}[i]$. Finally, Algorithm 3.1 (or Algorithm 3.2) is called with these adjusted mask components and bootstrapping keys.

A.2 Rounding to multiplicative (sub-)groups

A different strategy, described in [LMK⁺23, Section 3.3], involves modifying the modulus switching so that its outputs directly lie in $(\mathbb{Z}/2N\mathbb{Z})^\times \cup \{0\}$. This

¹¹ In [LMK⁺23, Algorithm 4], this is a post-processing step due to the initial value of the accumulator to $\tau_{-g}(x^{-\tilde{b}} \cdot v(x))$. In the setting of Algorithm 3.2 though, this can be readily moved in the “pre-conditioning” phase.

rounding to odd values (or to zero) can be expressed as¹²

$$\lfloor x \rfloor_{\text{odd}} := \begin{cases} 0 & \text{if } |x| \leq \frac{1}{2}, \\ 1 + 2 \cdot \lfloor \frac{x}{2} \rfloor & \text{otherwise.} \end{cases}$$

Hence, the modulus switching of an LWE ciphertext $(a_1, \dots, a_n, b) \in (\mathbb{Z}/q\mathbb{Z})^{n+1}$ with the above rounding operator outputs $(\tilde{a}_1, \dots, \tilde{a}_n, \tilde{b})$ with

$$\tilde{a}_i = \left\lfloor \frac{2N}{q} a_i \right\rfloor_{\text{odd}} \text{ for } i \in \llbracket 1, n \rrbracket, \text{ and } \tilde{b} = \left\lfloor \frac{2N}{q} b \right\rfloor.$$

This technique has been further extended in a recent work by Wang *et al.* [WWL⁺24] where the modulus switching gives elements of some predetermined cyclic subgroup of $(\mathbb{Z}/2N\mathbb{Z})^\times \simeq \langle \pm 1 \rangle \times \langle g \rangle$. Formally, for $\theta \in \llbracket 1, \log_2 N \rrbracket$ with $N \geq 4$, let G_θ be the following cyclic subgroup

$$G_\theta := \langle g^{2^{\theta-1}} \rangle = \{u \in (\mathbb{Z}/2N\mathbb{Z})^\times \mid u \equiv 1 \pmod{2^{\theta+1}}\} \simeq \mathbb{Z}/\frac{N}{2^\theta}\mathbb{Z}.$$

The corresponding rounding to the nearest element in $G_\theta \cup \{0\}$ is defined as

$$\lfloor x \rfloor_\theta := \begin{cases} 0 & \text{if } x \in \left[\frac{1}{2} - 2^\theta, \frac{1}{2}\right], \\ 1 + 2^{\theta+1} \cdot \left\lfloor \frac{x-1}{2^{\theta+1}} \right\rfloor & \text{otherwise.} \end{cases} \quad (\text{A.1})$$

Note $\lfloor x \rfloor_{\text{odd}} = \lfloor x \rfloor_0$ corresponds to rounding to the (however non-cyclic) entire group $G_0 := (\mathbb{Z}/2N\mathbb{Z})^\times$. The modulus switching then outputs

$$\tilde{a}_i = \left\lfloor \frac{2N}{q} a_i \right\rfloor_\theta \text{ for } i \in \llbracket 1, n \rrbracket, \text{ and } \tilde{b} = \left\lfloor \frac{2N}{q} b \right\rfloor.$$

Since G_θ is a sparser cyclic subgroup of G_0 of cardinal $\frac{N}{2^\theta}$, the mask values after modulus switching are concentrated on a smaller set. As a result, the number of automorphism key switches is reduced, as are the gaps between distinct values in terms of jumps by $g^{2^{\theta-1}}$. However, and crucially, the modified modulus switching procedure induces an increased drift, leading to a higher overall failure probability (see e.g., [WWL⁺24, Table 4]). This shall be accounted for when designing parameters and makes it a lot harder to get a fair comparison in terms of performance or key size between works such as [LMK⁺23] and [WWL⁺24]. We refer the reader to [BJSW25] for a detailed discussion on the drift.

B Correctness of Algorithm 3.1

For completeness, we provide a proof of the correctness of Algorithm 3.1 below. While the algorithm essentially builds on [LMK⁺23], the proof differs noticeably from [LMK⁺23, Section 3.1].

¹² Rounding small values to 0 is a free optimization using that null (more generally, small, see e.g., [HKLS24]) mask components can be skipped entirely in the blind rotation loop.

Proposition B.1. *Algorithm 3.1 is correct.*

Proof. Let $q_i = x^{-\tilde{b} + \sum_{j=1}^i \tilde{a}_j s_j} \cdot v$. For $t \in \llbracket 0, N/2 \rrbracket$, consider the polynomials $\tilde{q}_t^\pm \in \mathcal{R}_q$ recursively defined by $\tilde{q}_{N/2}^- = q_0 = v \cdot x^{-\tilde{b}}$, $\tilde{q}_{N/2}^+ = \tilde{q}_0^-$, and for all $N/2 > t \geq 0$,

$$\tilde{q}_t^\pm = \tilde{q}_{t+1}^\pm \cdot x^{\pm g^t \cdot \sum_{i \in I_t^\pm} s_i},$$

where all $\pm$ choices are identical. By Equations (3.1), (3.2) and (3.3), it follows that $\tilde{q}_0^+ = q_n$. We now inductively show that after each iteration of the respective loops on $I^\pm$, ACC contains a GLWE encryption of $\tau_{\pm g^{-t_{\text{old}}}}(\tilde{q}_t^\pm)$.¹³

First, ACC is initialized to a (trivial) GLWE encryption of $\tau_{-1}(q_0)$, which is indeed equal to $\tau_{-g^{-N/2}}(\tilde{q}_{N/2}^-)$. Inside the loop, the induction hypothesis is preserved as long as $I_t^\pm$ is empty, since in this case $\tilde{q}_t^\pm = \tilde{q}_{t_{\text{old}}}^\pm$. Assume now that $I_t^\pm$ is non-empty for some $N/2 > t \geq 0$ and ACC contains a GLWE encryption of $\tau_{\pm g^{-t_{\text{old}}}}(\tilde{q}_{t+1}^\pm)$. Then, τ_{g^δ} is homomorphically applied to ACC to obtain a GLWE encryption of $\tau_{\pm g^{-t}}(\tilde{q}_{t+1}^\pm)$, and the following external products finally yield as expected a GLWE encryption of

$$\tau_{\pm g^{-t}}(\tilde{q}_{t+1}^\pm) \cdot x^{\sum_{i \in I_t^\pm} s_i} = \tau_{\pm g^{-t}}(\tilde{q}_{t+1}^\pm \cdot x^{\pm g^t \sum_{i \in I_t^\pm} s_i}) = \tau_{\pm g^{-t}}(\tilde{q}_t^\pm).$$

After the first loop, ACC contains an encryption of $\tau_{-g^{-t_{\text{old}}}}(\tilde{q}_0^-)$, which after evaluating τ_{-g} on Line 9 becomes an encryption of $\tau_{g^{-t_{\text{old}}+1}}(\tilde{q}_0^-)$. Decreasing t_{old} by 1, this initializes correctly the second loop. Likewise, after the second loop ACC contains a GLWE encryption of $\tau_{g^{-t_{\text{old}}}}(\tilde{q}_0^+)$, implying the result since the last lines of the algorithm apply $\tau_{g^{t_{\text{old}}}}$. $\square$

C Comparison to Adaptive Key Update

To better emphasize the fundamental features of our automorphism-parametrized blind rotation, we mentioned Page 20 a generalization of [LLW⁺24, Lee24] using GLWE[®] ciphertexts of $\{x^{\pm s_i}, \dots, x^{\pm g^K s_i}\}$ for higher values $K \geq 0$. In this appendix, we present additional experimental results counting the number of automorphism key switches for this method, which demonstrate the clear and significant advantage of our approach.

This technique has been recently formalized in [ZWC25, Algorithm 6] under the name *Adaptive Key Update* (AKU). The authors use instead pre-rotated versions of GLWE[®] ciphertexts of x^{s_i} , but up to rewriting, both procedures are *mathematically* identical. To avoid confusion, we denote their parameter w by $(K+1)$ and restrict to the case $\theta = 1$ as in the rest of the paper.¹⁴ We also decoupled the window size (w'_{opt}) used for homomorphic automorphism

¹³ Actually, t_{old} encodes $\log_g \pm \tilde{a}_{\text{old}}$ where $\tilde{a}_{\text{old}}$ is as in Figure 2.1(b), with the masks $\tilde{a}_i$'s reordered according to $I_t^\pm$.

¹⁴ See the discussion at the end of Appendix A.2.

Table C.1: Experimentally measured number of regular ($\otimes$), automorphism-parametrized ($\otimes_S$) external products, and automorphism key switches (KS) for automorphism-based blind rotations, averaged over 10^4 samples. Note that in [ZWC25, Alg. 6], w corresponds to $(K + 1)$ here.

(a) Parameters $n = 465$ and $N = 1024$, $k = 1$.

Method	w'_{opt}	Keys ($\# \text{GLWE}^\nabla$)	$\#(\otimes)$	$\#(\otimes_S)$	$\#(\text{KS})$
AKU ([ZWC25, Alg. 6]), $K = 0$	8	$4n + 8$	465	0	305.6
AKU ([ZWC25, Alg. 6]), $K = 1$	8	$8n + 8$	465	0	191.5
AKU ([ZWC25, Alg. 6]), $K = 2$	8	$12n + 8$	465	0	139.6
AKU ([ZWC25, Alg. 6]), $K = 3$	8	$16n + 8$	465	0	109.8
AKU ([ZWC25, Alg. 6]), $K = 4$	8	$20n + 8$	465	0	90.6
AKU ([ZWC25, Alg. 6]), $K = 5$	8	$24n + 8$	465	0	77.0

(b) Parameters $n = 834$ and $N = 2048$, $k = 1$.

Method	w'_{opt}	Keys ($\# \text{GLWE}^\nabla$)	$\#(\otimes)$	$\#(\otimes_S)$	$\#(\text{KS})$
AKU ([ZWC25, Alg. 6]), $K = 0$	10	$4n + 10$	834	0	570.0
AKU ([ZWC25, Alg. 6]), $K = 1$	10	$8n + 10$	834	0	366.1
AKU ([ZWC25, Alg. 6]), $K = 2$	10	$12n + 10$	834	0	269.9
AKU ([ZWC25, Alg. 6]), $K = 3$	10	$16n + 10$	834	0	213.7
AKU ([ZWC25, Alg. 6]), $K = 4$	10	$20n + 10$	834	0	177.0
AKU ([ZWC25, Alg. 6]), $K = 5$	10	$24n + 10$	834	0	150.9

evaluations from K , as is the case in [LLW⁺24], to avoid artificial singularities in the number of key switches when K (i.e., $w - 1$ in [ZWC25]) is very small.

The results are reported in Table C.1 (compare with Table 5.4); notice that the numbers in the (a) part are consistent with [ZWC25, Figure 2]. This shows that with keys comprising as much as $24n$ GLWE^∇ ciphertexts, this method still requires 77 automorphism key switches. By contrast, our method achieves an average of 1.6 key switches using only $17n$ GLWE^∇ ciphertexts and already requires 30% less key switches (50.6) for over 3 times smaller keys ($7n$ GLWE^∇ ciphertexts). The “floor” behaviour identified in [ZWC25] reflects a fundamental limitation discussed on Page 20.